

Neural Style Transfer

Final Report

Final Year Project Preliminary Report (CM3015- Machine Learning and Neural Networks)

GitHub project link <https://github.com/nancyyrashed/graduation-project-final>

Github website link: https://github.com/nancyyrashed/full_nst_website

Website link: <https://neural-style-transfer-graduation-25.streamlit.app/>

(word count: 9996)

Contents

Neural Style Transfer Final Report.....	1
1. Introduction (word count: 523).....	4
2. Literature review (word count: 2356).....	5
3. Project design (word count: 1887)	13
3.1. OVERVIEW OF THE PROJECT.....	13
3.2. TEMPLATE USED	13
3.3. DOMAIN AND USERS.....	14
3.4. JUSTIFICATION OF DESIGN CHOICES.....	15
3.5. System Architecture	16
3.6. TECHNOLOGIES AND METHODS.....	17
3.7. EVALUATION AND TESTING PLAN	18
Qualitative Evaluation	18
3.8. WORK PLAN	19
3.9. CONTINGENCY PLAN.....	20
4. Implementation (word count: 2510).....	21
4.1. Overview of the project	21
4.2. Description of the features being implemented in the project	22
4.3 Explanation of the algorithms, techniques and methods used	24
4.4 Explanation of the important parts of the code	27
Gatys et al.	27
Johnson et al.....	29
AdaIN.....	31
Dumoulin et al	33
Dumoulin v2	35
Dumoulin V2 – multi-style.....	36
Dumoulin V2 – video style transfer.....	37
4.5 Visual representation of the results	38
1. gatys et al visual results	39

2. Johnson et al visual results	41
2.2. Learning progress of Johnson’s experiment six through the iterations:.....	42
3. AdaIN et al visual results:	43
3.2.	44
4.2. Learning progress of Johnson’s experiment seven through the iterations:	46
5. Dumoulin V2	47
6. Dumoulin’s V2 multi-style:	48
7. Dumoulin’s V2 video style transfer:	48
5. Evaluation (word count: 2145)	49
5.1. Qualitative Evaluation.....	49
User surveys:	49
5.2 Quantitative evaluation	92
1. content and style loss (MSE):.....	92
SSIM	94
DF/SF	95
LPIPS	96
Processing time	97
6. Conclusion (word count: 522).....	98
7. References	100

1.Introduction (word count: 523)

Neural Style Transfer (NST) is a groundbreaking and amazing technique in the computer vision field that enables to combine the content of a picture with the style of another one. These characteristics are put together in an optimization process to create a new image that preserves the content of one and the style of the other. The optimization process gradually reduces the gap between the loss of content and style to achieve a final output that is a creative combination of the original content and artistic style.

However, the creation of a single stylized image can be slow, taking minutes to hours on standard hardware, which restricts the use of the technique for real-time tasks, such as video style transfer or interactive design. The motivation of this project is not merely to learn the mechanism of NST process but also to streamline it to apply it in real life. This research aims to address the key limitations of computational inefficiency and make NST more practical for real-time purposes, as well as for applications beyond image processing, such as video production and real-time image manipulation.

This project follows Project Template 1: Neural Style Transfer from the CM3015 Machine Learning course, but goes beyond the work of Gatys et al. While the project starts with the original NST approach using VGG-19 for feature extraction and the reduction of content and style losses, it also builds on this method in several ways. It explores the integration of multiple styles simultaneously, providing a richer and more dynamic output. This multi-style transfer approach will test how various styles can be blended together and whether this affects the quality or performance of the output.

Moreover, the project also explores other convolutional neural network (CNN) structures, including VGG-16 and ResNet, to explore their effects on the quality of the stylized images and the processing time. These architectures may perhaps provide a more efficient trade-off between visual fidelity and computational efficiency, and their performance will be one of the points of interest.

The other important aspect of the project is the expansion of NST to the video domain. The problem of using NST on video is that it requires temporal coherence - the style should be the same across the frames, without flickering or incoherence. This project will address this issue and explore how video NST can be made feasible for real-time applications.

The project will evaluate the results both qualitatively and quantitatively. On the qualitative side, the visual quality of the generated images and videos will be assessed through subjective comparison. On a quantitative basis, the project will record the amount of time required to complete the stylization process and monitor the computational resources. It

will also compare the content and style fidelity and the performance of the default approach and more metrics we will explore later.

Ultimately, this project aims to push the boundaries of NST by improving its scalability, speed, and applicability to real-time scenarios. By testing different architectures, optimization techniques, and multi-style approaches, the goal is to make NST a more practical and accessible tool for creative professionals, particularly in video production, interactive media, and digital art.

2. Literature review (word count: 2356)

Neural Style Transfer (NST) is a leading technique blurring the lines between computer vision and image art by combining the look of one picture with the content of another. Immediately after Gatys et al. introduced NST in 2016, several methods focused on improving speed, flexibility and how closely NST can copy visual features of images. This report reviews important works in NST, ranging from fundamental optimizing strategies to real-time and feed-forward networks as well as methods for arbitrary style transfer. The goal is to critically assess the development of NST, bring out both its positive and negative points and encourage a project that applies them efficiently in an environment.

2.1. FOUNDATIONAL PAPER: GATYS ET AL. (2016) – NEURAL STYLE TRANSFER USING CNNs

The concept of Neural Style Transfer (NST) was first introduced in 2016 by Gatys, Ecker and Bethge in a paper that demonstrated a new way to merge the content of one image with the artistic style of another using the machine learning power of CNNs. It was found that VGG19, when used as a pretrained CNN on ImageNet, separates both content and style details into various layers. The deeper layers contain information about object shapes and spatial arrangement as high-level semantics representing the content of the image, while the style is represented by the correlations between feature activations, modeled by Gram matrices that capture texture and color patterns across multiple layers.

The algorithm can start with either a random noise image or a replica of the content image and it continuously updates the pixels values through the gradient descent, minimizing a composite loss function. This loss consists of a content loss, measuring the Euclidean distance between the feature maps of the generated image and the content image at deeper CNN layers, and a style loss, quantifying the difference between Gram matrices of the generated image and the style image

computed at multiple layers. By balancing these losses using chosen weights, optimization produces an image that displays the main features of the content image with the colors, patterns and textures coming from the style picture.

The approach leverages backpropagation to adjust the pixels of the generated image, effectively performing inverse optimization through the pretrained CNN's feature space. The process continues until convergence or until a visually satisfying result is produced, resulting in an image that convincingly fuses content and style in a way that was unprecedented for generative image manipulation at the time.

Critical Evaluation

The study by Gatys et al. introduced computer vision and artistic image synthesis to a new level by proving that CNNs intended for classification can also be used to create artworks. Thanks to its properties, the method can adapt to almost any photo to apply a different style, without needing to train for each style. The generated images display both the main content and attractive textures, often better than pictures made without using deep learning skills.

Still, this technique's biggest drawback is that it consumes significant computer resources. Because each output image requires a separate, iterative optimization process with many forward and backward passes through the CNN, generating a single stylized image takes minutes to hours on standard hardware, precluding real-time use. Since VGG-19 was developed specifically for classifying images, it may struggle to capture every detail in styles and spatial patterns. Additionally, the method's dependency on careful hyperparameter tuning (loss weights, initialization) can influence the quality and style fidelity, often requiring trial and error.

Motivation for project

This approach forms the basis for the entire project. Being aware of its workflows and possible limits, a person may highlight where changes and alternative paths could be added. Its computational expense motivates exploring more efficient NST methods, such as feed-forward networks or feature transformation techniques, which aim to produce stylized outputs in a single forward pass. Given that Google Colab does not have much GPU power or memory, the project must use resources efficiently. Also, investigating the influence of style-content loss weighting on output quality sheds light on controlling the style-content balance, a crucial factor for NST usability.

2.2. SPEED IMPROVEMENTS / PRACTICALITY: JOHNSON ET AL. (2016) – PERCEPTUAL LOSSES FOR REALTIME STYLE TRANSFER

While Gatys and his team created its theoretical foundation, the method is impractical because it takes too much time. According to Johnson, Alahi and Fei-Fei (2016), these authors introduced a new solution: a feed-forward neural network would be pre-trained for style transfer, so that the application could carry out the task in real-time. Instead of optimizing each image separately, their approach trains a transformation network that directly maps a content image to its stylized counterpart in a single forward pass.

Details extracted from a VGG-16 network with perceptual losses, much like Gatys et al.'s content and style losses, are the main innovation in this approach. These losses guide the training of the feed-forward network by enforcing similarity in high-level features and style statistics between the generated and target images, effectively learning to approximate the iterative optimization process. Since it is fully convolutional, the transformation network supports processing images with arbitrary resolutions.

Critical Evaluation

Johnson et al.'s method makes NST faster, we can now perform real-time stylization by using it on modern GPUs and mobile devices, thus making NST practical for interactive applications like video stylization, mobile apps, and augmented reality. With enough training and data, the style of the results can be as good as those from Gatys et al. At the same time, reaching higher speeds means less flexibility. Since every model is made for a specific style, creating new styles requires brand new training which makes the process less scalable. Training remains computationally intensive but is a one-time offline process. Besides, the quality of the network's output strongly depends on the chosen data set for training and on the values of its hyperparameters.

Motivation for project

Since the project aims for efficiency and scalability, this work is very relevant. It illustrates the trade-off between flexibility and performance, so the research focuses on models that stay quick while being able to adapt to different styles. As training with Google Colab is slow and has fewer resources, it is important to understand the pros and cons of feedforward networks. Also, assessing feed-forward against optimization-based approaches with respect to hyperparameters

gives insight into their impact on style fidelity and computational overhead, guiding method selection.

2.3. ARBITRARY STYLE TRANSFER: HUANG & BELONGIE (2017) – ADAPTIVE INSTANCE NORMALIZATION (ADAIN)

Adaptive Instance Normalization (AdaIN) was suggested by Huang and Belongie (2017) to allow any type of style transfer, without needing to train the model again. AdaIN sets the deep features' mean and variance of the content image to match the mean and variance of the style image. This moment-matching is applied via instance normalization layers, which modulate the content features to adopt the style statistics.

The architecture employs a pretrained VGG-19 encoder to extract features, followed by AdaIN layers that adjust the feature statistics, and a decoder network trained for image reconstruction from the adjusted features. AdaIN simply changes the characteristics of the content throughout the process to match the statistics of any chosen style, allowing the model to transfer styles with just one forward pass.

Critical Evaluation

AdaIN is a valuable step forward because it makes style transfer flexible and fast for delivering arbitrary style transfer efficiently. Its single-pass inference supports real-time applications without the need for retraining each time. Yet, focusing on the mean and variance through AdaIN may make it difficult for the method to represent complex style aspects, including detailed textures or spatial relationships. Thus, at times the approach does not create as detailed or varied results as techniques that deal with statistics or patch information.

Motivation for Project

Indeed, AdaIN contributes to the main goal of the project: speed, flexibility and efficiency within limited computational constraints. Because the method misses some stylistic information, people often seek out approaches that use more information, for example, second-order statistics, patch-based matching, or learned feature transforms so as to achieve the best balance of quality and efficiency.

2.4. A LEARNED REPRESENTATION FOR ARTISTIC STYLE: DUMOULIN ET AL. (2017) – CONDITIONAL INSTANCE NORMALIZATION (CIN)

Dumoulin et al. (2017) introduced Conditional Instance Normalization (CIN), a method for efficient style transfer that can handle multiple styles using a single model. Unlike traditional methods, which require separate models for each style, CIN learns style-specific parameters (γ and β) during training, making it memory efficient while maintaining shared convolutional weights across styles. This technique is particularly advantageous when combining multiple styles into one stylized output.

CIN applies instance normalization conditioned on the style of the image. The method achieves impressive style fidelity, handling diverse artistic styles by capturing texture and color variations. It provides a scalable solution for artistic style transfer, with the ability to blend multiple styles, making it more flexible than single-style transfer methods.

Critical Evaluation

Dumoulin et al.'s approach offers a major advantage in memory efficiency and computational cost compared to training multiple separate networks. It excels in transferring the core features of various styles while preserving content integrity. However, it occasionally struggles to capture highly intricate textures or fine details of complex artistic styles, leading to slight distortions in some cases.

The trade-off between model generality and style representation accuracy can affect the quality of style transfer for certain highly detailed styles. Despite this, the model is highly versatile and offers a good balance between efficiency and style fidelity, particularly for real-time applications or environments with limited resources.

Motivation for Project

The Dumoulin model provides a robust and scalable framework for style transfer, making it suitable for real-time applications and diverse artistic styles. Its ability to combine multiple styles within a single network without sacrificing computational efficiency makes it a strong foundation for future style transfer research, especially in constrained resource environments.

2.5. ALTERNATIVE APPROACH: REAL-TIME NEURAL STYLE TRANSFER FOR VIDEOS (HUANG ET AL., 2020)

Another variant of Neural Style Transfer (NST) is described in the article "Real-Time Neural Style Transfer for Videos" (2020). This method proposes real-time video style transfer and deals with the key problem of preserving the temporal coherence of video frames in style transfer. Compared to the conventional optimization-based approaches to NST, which mainly emphasize the content loss and style loss as the goals of the transfer, the presented approach adds a temporal filter to the idea of style transfer, making the stylized videos consistent in style and free of artifacts like flickering

Here, the authors present a generator network that learns to translate content images into their stylized counterparts, as in standard NST. However, their approach incorporates the concept of temporal consistency into the framework, so that the visual style is consistent across video frames. This is especially important in real-time tasks like interactive media, video production, and augmented reality (AR). In these cases, the visual style should stay consistent in-between video frames.

Critical Evaluation

The effectiveness of this method is in the fact that it allows achieving real-time video stylization, which makes it possible to transfer the style across the video frames with high quality. The technique has temporal coherence, which is normally compromised in other NST models, thus it is highly applicable in dynamic content. However, the technique still consumes a lot of computation resources, and though it is in real time, it can still optimize its efficiency. Also, the method uses a temporal filter to be consistent, this adds complexity in the balance between spatial style accuracy and temporal consistency.

Project Motivation

Such an approach fits the purpose of the project to improve real-time style transfer, in particular video. This addresses a long-standing issue in video style transfer by focusing on the problem of temporal coherence. Interactive media, video streaming, and augmented reality all depend on the possibility to maintain stylistic consistency between video frames. By incorporating real-time video style transfer into the project, it will be more applicable in fields where video material must be styled on the fly without compromising the visual quality of the result or introducing the flickering artifacts.

2.6. A COMPREHENSIVE REVIEW OF NEURAL STYLE TRANSFER: JING ET AL. (2019)

They review various NST technologies up to 2019 and sort them into categories known as optimization-based, feed-forward networks, arbitrary style transfer and universal style transfer frameworks. They study the advantages and disadvantages, choose several ways to evaluate and focus on the open issues like semantic content preservation, computational efficiency, and robustness to style diversity.

They include major works like Gatys et al., speed-optimized networks such as Johnson et al. and methods for feature transformation (WCT and AdaIN), putting the main emphasis on the disagreements between style fidelity, maintaining content, and being efficient. It is also observed the difficulty of quantitatively evaluating NST due to its subjective, artistic nature, calling for better benchmarks and standardized protocols.

Critical Evaluation

The document is very useful as it explains the growth of NST, links different approaches and points out future areas that need exploration. Nonetheless, it cannot go into great detail about various models or new ways of conducting experiments. It mainly compiles other researchers' findings, unlike introducing new information from studies.

Motivation for Project

The review explains how the research questions for the project are related to the NST field and points out that attention should be given mainly to efficiency, flexibility and evaluation methodology. It underscores the importance of hyperparameter tuning and it brings attention to how NST results should be assessed, using both qualitative and quantitative assessment of NST results, helping align the project's approach with contemporary standards and recognized gaps.

2.7. CONCLUSION

I tried to add value beyond summarising the different work coherently, I synthesized and linked literature directly to my project by grouping papers by evolution, comparing approaches.

This survey outlined the major developments in Neural Style Transfer (NST), starting with optimization-based approaches to real-time applications, such as feed-forward networks and arbitrary style transfer. These advances have paved the way to NST, which have solved the problems of computational inefficiency and style

fidelity. As the advancements, the present project will focus on new adaptations, like trying out optimizers, style models, and CNN structures, to enhance the efficiency and scalability of NST with a view of preserving high-quality output in real-time applications.

Paper	Method	Style Transfer Type	Speed (FPS)	Content/Style Loss	Notes
Gatys et al. (2016)	Optimization based	Per-style	~0.5	Lowest content loss, highest style fidelity	Very slow; not suitable for real-time
Johnson et al. (2016) (<i>Fast Style Transfer</i>)	Feed-forward CNN	Per-style	25-30	Moderate content loss, good stylization	Real-time capable, trained per style
Huang & Belongie (2017) (<i>AdaIN</i>)	Adaptive Instance Norm	Arbitrary	30	Slightly higher content loss than WCT	Excellent speed, stylization quality varies
Jing et al. (2020) (<i>Survey Paper</i>)	Review Paper	N/A	N/A	N/A	Summarizes NST approaches, no new metrics
Dumoulin et al. (2017) (CIN)	Conditional Instance Normalization	Arbitrary	~2	Lower content loss than AdaIN	High style fidelity but slower
Huang et al. (2020) (Real-Time Neural Style Transfer for Videos)	Temporal Filter + CNN	Arbitrary	~30-60	Varies	Real-time video style transfer with temporal coherence, suitable for dynamic applications

Table 1. Comparison of key Neural Style Transfer methods, showing their style transfer type, speed, content/style loss, and notable characteristics.

3. Project design (word count: 1887)

3.1. OVERVIEW OF THE PROJECT

In this project we will improve the Neural Style Transfer (NST) approach and optimize it to run in real-time. Originally introduced by Gatys et al. (2016), NST separates the content of an image and the style of another one and recombines them with the help of Convolutional Neural Networks (CNNs). The default approach yields high-quality results, but it is too computationally expensive to be used in real-time applications, like making videos or interactive media.

This project is all about enhancing the speed and the scalability of NST, without compromising on artistic fidelity. The first implementation will entail the initial deployment of the original NST algorithm with VGG-19 as the feature extractor of the content and style. At that point, the project will consider optimizations, and CNN architectures (e.g., VGG-16, ResNet) to balance between computing and output quality.

Further, this project will explore advanced functionalities, such as multi-style transfer, enabling the combination of multiple styles within a single image. Furthermore, the project will expand NST to video style transfer, ensuring temporal coherence between video frames to eliminate flickering. Its purpose is to enable NST to be relevant to real-time interactive applications, such as augmented reality and video games, where speed and high visual quality are critical.

3.2. TEMPLATE USED

The project uses the Machine Learning and Neural Network Neural Style Transfer (NST) template, focusing on utilizing deep learning and machine learning methods in increasing efficiency and scale of style transfer. The template is based on learning image and video features, and the loss of content and the loss of style are kept minimal to produce high quality results. The main consideration is to enable NST to be used in real-time applications, and this needs to optimize both speed and resource consumption.

3.3. DOMAIN AND USERS

Domain

The domain of the project is at the confluence of computer vision and digital art, namely Neural Style Transfer (NST). NST is a method that combines the content of one image with the style of another, and its area of application is very broad in terms of art generation, image manipulation, and visual content creation. Since the project will be optimizing NST to be used in real-time, the domain also includes video production, augmented reality (AR), virtual reality (VR), and interactive media, where real-time style transfer can be used to apply artistic filters or improve the visual content

Users

The primary users of the technology developed in this project span multiple industries and fields, each with different needs:

Digital Artists: Artists looking to make original pieces through the novel rearrangement of content and style will find useful the fact that NST can create a wide variety of artistic outputs fast and efficiently.

Video Game Developers: The game designers can use NST to produce stylized level designs and character models which can be dynamically react to the situation in game, enhancing the overall visual impression of the game.

Film Producers and Editors: the real-time capabilities of artistic effects on videos will help film producers to develop visually special effects in films, documentaries or animation.

Interactive Media Developers: In the case of those who work in the field of augmented reality (AR) and virtual reality (VR), NST could be used to develop immersive, interactive spaces in which the user can modify or affect the visual display in real time.

Creative Technologists: Developers working on interactive installations or performance art will be able to run real-time stylization on live media, such as a projected image or a live broadcast, giving them more freedom to be creative.

Students and Researchers: The project will also be used by students and researchers looking into computer vision, machine learning, and AI ethics topics. The project briefly mentions both the technical side of NST as well as its ethical

implications, especially regarding AI and its effects on creativity and the responsible design of AI.

3.4. JUSTIFICATION OF DESIGN CHOICES

The project is driven by some of the major gaps observed in the literature regarding the Neural Style Transfer (NST):

Computational Inefficiency: The conventional NST algorithms, including the one proposed by Gatys et al. (2016), are computationally costly because they rely on iterative optimization. This makes the method not applicable in real time such as video processing and interactive media. To work around it, the project uses feed-forward networks (Johnson et al. 2016), requiring no iterative optimization, saving a lot of processing time. That enables real-time NST to be achieved without compromising the artistic content of the result.

Inability to be flexible with Style Transfer: NST models, such as those introduced by Gatys et al. (2016) and Johnson et al. (2016), usually transfer one style to the content image. To improve artistic flexibility, this project investigates into a variety of style models, each of which the user can combine with other styles in use. The adaptation increases the dynamic application capability of NST.

Temporal Coherence in Video Style Transfer: Existing NST algorithms fail at video style transfer due to the frequent occurrence of flickering or disparities between neighboring video frames. Huang et al. (2020) focus on this problem, proposing temporal coherence in video style transfer, where the visual style is consistent between frames. The project takes these observations into account so that the style transfer of videos is consistent and fluent, which is appropriate to use in tasks like video editing.

Generalization to Different Content Types: Most NST models achieve good results on one type of content but fail on transferring the style to a very different image, i.e. between architectural buildings and natural scenery. The current project faces this issue by employing AdaIN (Huang & Belongie, 2017) and Dumoulin new version I implemented to enable arbitrary style transfer and makes sure that the model can be generalized to various forms of content. It is essential when it comes to such applications as film production, video games, and digital art where a huge diversity of visual content must be stylized in a smooth manner.

3.5. System Architecture

The following elements are part of the project's modular architecture:

Input Processor: This component is used to load images/videos, resize and normalize them with the help of OpenCV as the data has to be ready to extract features.

Feature Extractor: Takes as input a pretrained VGG-19 model (Gatys et al., 2016) or VGG-16 and extracts high-level content features and low-level style features of the content and style images or video frames, with special handling for multi-style and unseen style images.

Style Transfer Models: Applies six NST architectures:

Gatys et al. (2016): VGG-19 optimized fine-grained, which generates high-quality results but is slower.

Johnson et al. (2016): Efficient, real-time feed-forward networks of fast stylization with moderate content loss

Huang Belongie (2017): AdaIN: arbitrary style transfer, flexible and efficient style transfer.

Dumoulin et al. (2017) - CIN: CIN model proposes Conditional Instance Normalization (CIN): This model effectively transfers styles by learning style-specific parameters (γ and β). With this approach, style transfer is flexible and can be done using a single model instead of using different models to represent different styles.

Dumoulin et al. (2017) - CIN Version 2 (Unseen Styles): A new version of CIN which can deal with unseen styles. This version is able to dynamically learn how to adjust to new styles without having to retrain

Dumoulin et al. (2017) - CIN Version 2 (Multi-Style): This version of CIN Version 2 is extended and allows multi-style transfer, which allows mixing two styles in one picture.

Video Stylization: applies Neural Style Transfer (NST) to each frame of the video individually using the trained Dumoulin V2 model, which allows for the stylization of previously unseen style images. Combines methods of Huang et al. (2020) to achieve temporal consistency in videos, eliminating flickering, and frame-to-frame consistency of the style transfer.

Output Reconstructor: Assembles the processed frames back into images or videos with FFmpeg, making sure that both stills and video look great.

Evaluation & Accessibility Tools: Side-by-side comparisons, User study/feedback and perceptual similarity metrics (SSIM), DF (Distance Function) and SF (Similarity Function) and LPIPS (Learned Perceptual Image Patch Similarity) are provided to evaluate and assess quality.

The entire project is coded in Python, executed in google colab, and managed in version control by GitHub.

3.6. TECHNOLOGIES AND METHODS

Tool/Method	Purpose
FFmpeg	Frame-to-video reconstruction and exporting video outputs
TensorFlow/Keras	Model definition and training
VGG-19, VGG-16	Feature extraction
Matplotlib/Seaborn	Data visualization for performance metrics and comparisons
GitHub	Version control and collaboration tracking
OpenCV	Image/video frame handling (loading, resizing, and normalization)
SSIM	Perceptual similarity measurement for content and style fidelity
LPIPS	Perceptual similarity using deep feature space for evaluating quality
DF (Distance Function) and SF (Similarity Function)	Evaluating style and content transfer quality using feature distance and similarity
ImageIO	Reading and writing images and videos, especially for GIFs and video processing
PyTorch	Core framework for implementing models and performing style transfer
Scipy	Optimization (fmin_l_bfgs_b for minimizing loss)
PIL (Pillow)	For image loading and manipulation (e.g., resizing, converting to tensor)

Torchvision	For image processing, including the pre-trained VGG16 model for feature extraction.
TQDM	For showing the progress of iterations during training with a visual progress bar.

Table 2. Tools and Methods Used in the Neural Style Transfer Implementation

3.7. EVALUATION AND TESTING PLAN

Qualitative Evaluation

Visual Comparison: Conduct side-by-side assessments of the original content and the stylized output to evaluate the preservation of content and the accuracy of the artistic style.

User Feedback: Participants will be selected from a mix of digital artists, designers, and users with varying expertise, ensuring diverse feedback. They will be contacted through email or social media platforms. In the evaluation, users will rate the aesthetic appeal, style consistency, and realism of the final images using a scale. The feedback will be collected via an online survey, allowing both quantitative ratings and qualitative comments.

Quantitative Evaluation

Content and Style Loss (MSE): Measures the difference between the content and style of the produced output and the inputs. This measure is critical to the evaluation of the style transfer accuracy.

Processing Time: The amount of time required to render the stylized image or video. This would be essential towards achieving real-time performance of the system.

Perceptual Similarity (SSIM): Evaluates the appearance of the stylized results based on their structural similarity to the input images.

Distance Function (DF) and Similarity Function (SF): Evaluate the quality of style and content transfer by comparing feature distances and similarity across different scales, capturing the structural consistency and texture preservation in the stylized output.

Learnable Perceptual Image Patch Similarity (LPIPS): Measures perceptual similarity using deep feature space, with lower LPIPS values indicating higher perceptual similarity between the content and stylized images.

Video Style Transfer Evaluation

Temporal Consistency: Analyzes the quality of style consistency frame by frame of the video, indicating a high-quality transition between frames and the absence of such problems as flickering (Huang et al., 2020).

Visual Artifacts: Monitors the final video output for inconsistencies or flickering that may arise during the stylization process.

Usability and Accessibility

Real-time Stylization: Test the system's ability to perform real-time style transfer in interactive contexts, such as augmented reality (AR) and video games, while maintaining low latency. I created a website using streamlit that will serve as the primary platform for users to experience and interact with the NST model in real time.

Accessibility: Ensure that contrast-enhanced versions of the stylized outputs are clearly visible and understandable for users with different visual impairments.

Cultural and Ethical Evaluation

Ensure that all artistic styles used are open-license and that proper citations and attributions are made for any non-original code or assets.

3.8. WORK PLAN

Phase	Timeline	Deliverables
Discovery	April 2025	Research, proposal video, literature review
Design	April – May 2025	Project design, work plan, evaluation plan
Development	May – July 2025	First prototype, video NST, testing several approaches
delivery	August – September 2025	Final report, presentation video, final prototype, evaluation

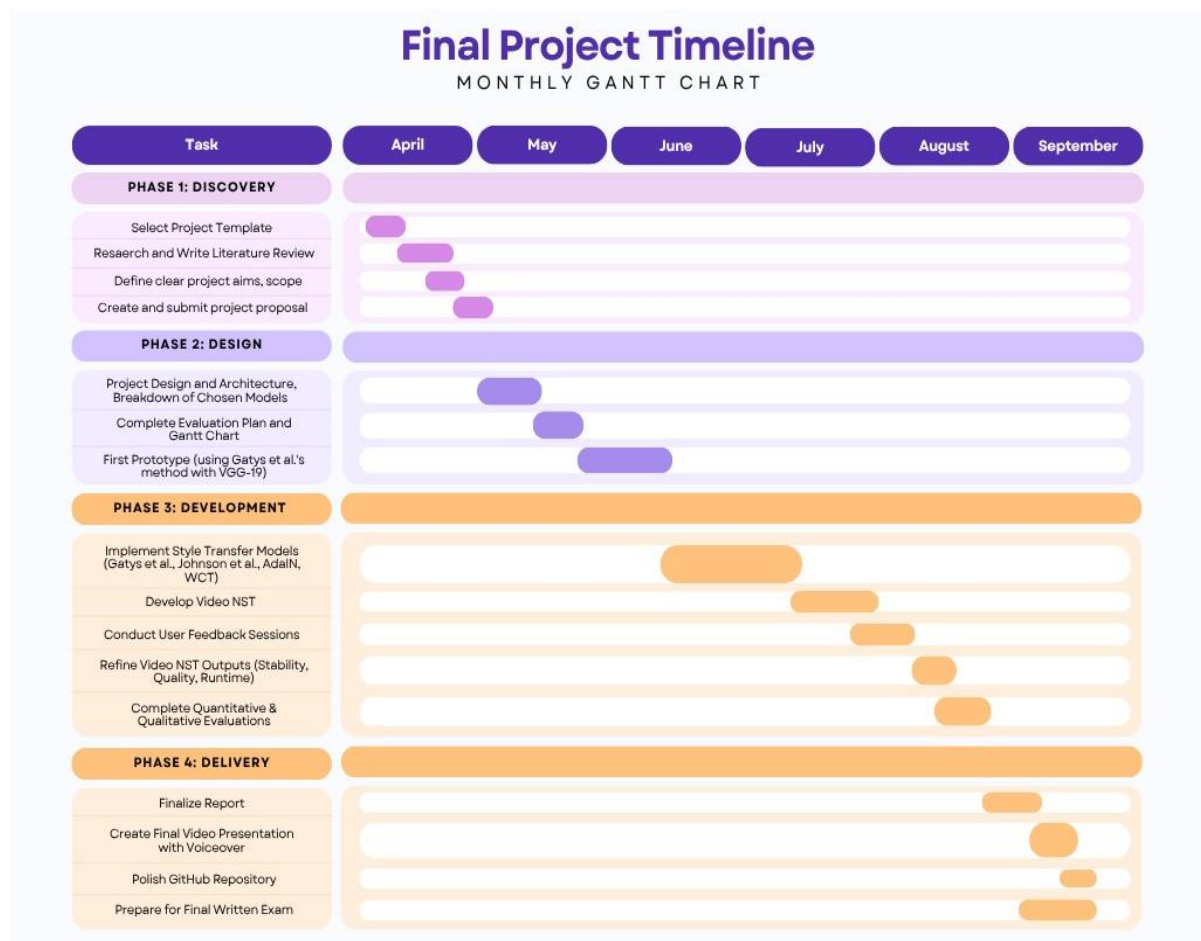


Figure (1) Gantt Chart of The Project across four phases: Discovery, Design, Development, and Delivery, from April to September 2025.

3.9. CONTINGENCY PLAN

In the event of performance problems, the following changes will be made:

Shorter Video Clips: To deal with the processing time and to transfer the real time video effectively, the project can consider changing to shorter video clips or smaller batches during the testing phase to make sure the system is not overloaded and causing undue delay.

Model Optimization: In case the processing time is still too big, attempts to optimize the models (resize images, simplify style models (fewer styles or layers)) will be tried to achieve a reasonable decrease in speed at the cost of not compromising too much on quality.

Fallback to Stable Models: When temporal coherence or flickering in video is problematic, a slower, stable model, such as Gatys et al. (2016) on certain frames, can be used to guarantee stability.

4.Implementation (word count: 2510)

4.1. Overview of the project

This project focuses on improving Neural Style Transfer (NST) algorithms by combining four established techniques Gatys et al. (2016), Johnson et al. (2016), AdaIN (Huang & Belongie, 2017), and Dumoulin et al. (2017) and introducing three novel extensions to enhance the quality and efficiency of NST for real-time applications, such as video processing and interactive media.

The established methods provide a foundation ranging from high-quality but computationally expensive iterative optimization (Gatys) to real-time capable feedforward networks (Johnson, AdaIN, Dumoulin). Each method contributes unique strengths: Gatys provides exceptional quality through VGG-19 feature extraction and iterative optimization; Johnson introduces feedforward efficiency; AdaIN enables arbitrary style transfer through adaptive normalization; and Dumoulin learns artistic style representations for multiple style handling.

In addition to these existing methods, this project introduces three novel extensions. Dumoulin V2 builds upon the original Dumoulin work by incorporating both Conditional Instance Normalization (CIN) and Adaptive Instance Normalization (AdaIN) to improve generalization to unseen styles while maintaining efficiency. Dumoulin V2 Multi-Style extends the V2 model to enable simultaneous application of multiple styles to different regions of an image, dividing content into two zones for distinct stylistic treatment. Dumoulin V2 Video Transfer adapts the V2 architecture for real-time video processing, using optical flow to ensure temporal consistency and prevent flickering artifacts between frames.

To optimize performance across all methods, eight comprehensive hyperparameter explorations were conducted for Gatys, Johnson, AdaIN, and the original Dumoulin models. The explored hyperparameters included content, style, total variation weights and other optimization parameters. The Dumoulin V2 variants utilized optimized parameters derived from the original Dumoulin exploration, allowing focus on the novel architectural improvements rather than hyperparameter tuning.

All methods are integrated into Streamlit-based web platform, enabling systematic comparison of NST techniques and providing a practical interface for both research evaluation and real-world applications.

4.2. Description of the features being implemented in the project

The original NST method proposed by Gatys et al. (2016) extracts both content and style features using the VGG-19 network. The content features are derived from the original content image, while the style features are extracted by calculating the Gram matrix of the style image. The optimization process minimizes the loss between the generated image's content and the target style, resulting in an image that combines both the content of the input image and the style of the reference image. While effective in producing high-quality outputs, this approach is computationally expensive due to its iterative optimization process.

The Fast Style Transfer implementation prioritizes speed while maintaining reasonable quality through a feedforward CNN architecture. The network consists of encoding layers that compress input images, residual blocks that perform style transformation, and decoding layers that reconstruct the final stylized output. Instance normalization layers throughout the network enable effective alignment between style and content features during the single forward pass. Each model is trained in one specific artistic style, requiring separate models for different styles but enabling real-time performance. The system generates stylized images in seconds, making it suitable for interactive applications where immediate feedback is essential.

The AdaIN implementation offers the most flexible style application by enabling universal style transfer with any style image without requiring style-specific training. The method uses adaptive instance normalization to dynamically match the statistical properties of content features to those of style features, specifically aligning the mean and variance values. The architecture employs a pretrained VGG-19 network as an encoder for feature extraction, combined with a lightweight decoder for image reconstruction. This approach combines the computational efficiency of feedforward networks with the flexibility of arbitrary style application, allowing users to apply any style image to any content image in real-time without preprocessing or model retraining.

The original Dumoulin implementation enables efficient multi-style handling through a conditional instance normalization approach. The model is trained on a dataset of content and several style images, learning how to apply styles more effectively while preserving the content's structure. It provides a system that can adapt to various kinds of artistic styles but not unseen styles.

The Dumoulin V2 extension improves upon the original implementation by incorporating a hybrid normalization approach that combines Conditional Instance Normalization (CIN) with Adaptive Instance Normalization (AdaIN). This hybrid system enables the model to handle both predefined styles through CIN and arbitrary unseen styles through AdaIN integration. The enhanced architecture better preserves fine details and achieves superior content-style balance compared to the original method. The flexible design allows users to apply to completely new styles not encountered during training, providing both the efficiency of conditional normalization and the versatility of adaptive normalization in a single unified system.

The Dumoulin V2 Multi-Style variant enables sophisticated style control by allowing the simultaneous application of multiple styles to different regions of an image. The model utilizes a multi-style approach that divides the content image into two regions and applies different styles to each region, preserving the overall structure while introducing diverse stylistic elements. The architecture consists of specialized convolutional layers designed to handle multiple style representations in parallel, ensuring smooth transitions between different style zones.

The Dumoulin V2 Video Transfer extension builds on the Dumoulin V2 architecture to enable real-time video style transfer. This method addresses the challenges of temporal consistency between video frames by using optical flow, which helps ensure smooth transitions between consecutive frames. By applying style transfer to each frame while maintaining temporal coherence, this extension prevents flickering and other artifacts that can occur in video-based NST.

All methods are integrated into a single platform called streamlit, allowing users to compare and select the most appropriate technique for their specific use case.

4.3 Explanation of the algorithms, techniques and methods used

The methods are described below along with the techniques and algorithms used in each method in the papers and code:

Gatys et al.(2016)NST Algorithm: The Gatys algorithm is based on Neural Style Transfer using a pre-trained VGG-19 network to extract features of content and style images. The content loss is measured by taking the difference between the high-level features of generated image and the content image. The style loss is obtained by comparing Gram matrices of the style image and generated image, which captures the correlation between feature maps in different channels. Both content and style features are extracted using the VGG-19 model, with comparison performed using Mean Squared Error (MSE) loss function. L-BFGS-B, a quasi Newton method, is used in iteratively optimizing the generated image through a weighted loss combining content loss, style loss and total variation loss.

Johnson, et al. (2016) Fast Style Transfer: Johnson et al. (2016) approach presents a more time-efficient scheme to carry out style transfer by creating a feedforward convolutional network which creates stylized images in one propagation mode and averts the requirement of iterative optimization. This kind of training is fed on content images instead of a combination of content and style images.

Within the code, definition of a CNN within ImageTransformationNet class defines convolution layer followed by several residual blocks. It has a network which learns a mapping between content images and their stylized versions and does so by minimizing the content dissimilarity and style dissimilarity during training. The instance normalization layer normalizes individual feature maps without depending on any other feature map making it possible to map the style of the reference image onto the content image.

The content and style losses in this approach are calculated by computing L2 distance between the content features and by calculating the difference between the Gram matrices of style image and generated image, respectively. Residual block allows us to train the network to learn efficient transformations.

AdaIN (Huang & Belongie, 2017): AdaIN makes the mean and variance of the content features correspond to the style features, providing faster processing with no optimization or iteration involved.

The class VGGEncoder in the given code extracts features based on VGG-19 model, which is used to calculate content and style features. The method used by AdaIN class is applying the method of adaptive instance normalization to match the

content features with the style features in terms of their mean and the variance. The Decoder class is then used to amalgamate the stylized image based on these features. The approach does not require any kind of iterative optimization, and hence, produces real-time style transfer.

It normalizes the style features so that the means and the variances are the same as the content features. This network has the ability to transfer the style in one pass, ideal to apply to real-time scenarios.

Dumoulin et al. (2017) Learned Representation: This method presents learned representation in artistic style transfer where the representation of artistic style is learned rather than using pretrained feature extractors. The CIN (Conditional Instance Normalization) module maps style codes in the content features. The StyleTransferNetwork class integrates convolutional layers and instance normalization to learn how to transfer style to content images. The model learns to transfer content images to stylized images by learning the association between content features and style codes. MSE between feature activations calculates content and style losses, with style loss computed using Gram matrix calculations.

Dumoulin V2: introduced an innovative approach for artistic style transfer by utilizing a hybrid method that combines Conditional Instance Normalization (CIN) with Adaptive Instance Normalization (AdaIN) to dynamically apply style transfer to content images. Unlike traditional methods that rely on pretrained feature extractors like Gram matrices, Dumoulin V2 learns the best representation for transferring artistic styles to content images. This hybrid approach enables more flexibility in adapting to various artistic styles.

The model is trained to learn a mapping between the content features and style codes, allowing it to handle a wide range of styles. The CIN component adjusts the feature maps according to the style code during the normalization, while AdaIN helps match the content features's mean and variance to those of the style image. This combination allows for efficient and flexible style transfer adaptable to complex, diverse styles.

The network learns to apply style to content images by calculating the MSE loss between content and style features, with style loss computed using the Gram matrix of features at different layers.

Dumoulin V2 Multi-Style: This variant applies two different styles to a single content image, using Conditional Instance Normalization (CIN) and Adaptive Instance Normalization (AdaIN). The content image is split into two halves—Style 1 is applied to the

left half, and Style 2 is applied to the right half. A blend zone is used to ensure a smooth transition between the styles.

The style statistics (mean and variance) for each style image are extracted separately and used to adjust the normalization of the feature maps for each half of the content image. The model minimizes content loss (measured by feature map similarity), style loss (using Gram matrices), and total variation loss to generate a blended image that retains the original content while combining both styles. The final output is an image with a seamless transition between two distinct styles.

Dumoulin V2 Video: approach adapts the original Dumoulin V2 method to work with video frames, applying style transfer while ensuring temporal consistency across frames. This version of Dumoulin V2 uses (CIN) to apply style transfer to each frame of the video, while also considering the relationship between consecutive frames.

The process works by applying the style transfer to each individual frame using the trained Dumoulin V2 model, ensuring that the content features of the frames remain intact while the style features are transferred. The key difference in the video version is the use of optical flow to ensure smooth transitions between consecutive frames. This prevents abrupt changes or flickering between frames, a common issue in traditional frame-by-frame style transfer.

Optical flow is calculated between consecutive frames to detect pixel movement, and this information is used to warp the previous frame's stylized image to match the current frame's content. The previous frame's stylized image is blended with the current frame to maintain temporal coherence. The resulting stylized frames are then compiled into a video, preserving both the artistic style and the continuity of motion across the frames.

4.4 Explanation of the important parts of the code

Gatys et al.

1. VGG-19 Feature Extraction Setup: This creates the perceptual feature extractor that enables content-style separation. Using frozen VGG-19 weights, it extracts content features from deep layers and style features from multiple layers. This multi-layer extraction is the foundation that makes neural style transfer possible.

```
content_layer_name = 'block5_conv2'

style_layer_names = ['block1_conv1', 'block2_conv1', 'block3_conv1', 'block4_conv1',
                    'block5_conv1']

vgg_model = vgg19.VGG19(weights='imagenet', include_top=False)
vgg_model.trainable = False

layer_outputs = [vgg_model.get_layer(name).output for name in style_layer_names +
                 [content_layer_name]]

feature_extractor = tf.keras.Model(inputs=vgg_model.input, outputs=layer_outputs)
```

2. Gram Matrix Calculation for Style Loss:

The Gram matrix captures style by computing correlations between feature channels, encoding texture patterns independent of spatial location. By reshaping feature maps and computing their matrix product, it represents "what textures co-occur" rather than spatial arrangement. This mathematical insight enables separating style from content and is the core innovation that made neural style transfer possible.

```
def compute_gram_matrix(tensor):  
    tensor = tf.transpose(tensor, (2, 0, 1))  
    features = tf.reshape(tensor, (tf.shape(tensor)[0], -1))  
    return tf.matmul(features, tf.transpose(features))  
  
class StyleLoss(tf.keras.layers.Layer):  
    def call(self, style_image, combined_image):  
        style_gram = compute_gram_matrix(style_image)  
        combined_gram = compute_gram_matrix(combined_image)  
        return self.weight * tf.reduce_sum(tf.square(style_gram - combined_gram)) / (4.0 *  
        (channels ** 2) * (size ** 2))
```

Johnson et al

1. VGG16 Feature Extractor: The VGG16 feature extractor uses a pre-trained VGG16 model to capture content and style features from layers like relu1_2, relu2_2, and relu3_3. These features are key for calculating content and style losses in Neural Style Transfer (NST).

```
class VGG16FeatureExtractor(nn.Module):

    def __init__(self):

        super(VGG16FeatureExtractor, self).__init__()

        vgg = models.vgg16(pretrained=True).features # Load pre-trained VGG16 model

        self.relu1_2 = nn.Sequential(*vgg[:4]) # Extract features from relu1_2 layer

        self.relu2_2 = nn.Sequential(*vgg[4:9]) # Extract features from relu2_2 layer

        self.relu3_3 = nn.Sequential(*vgg[9:16]) # Extract features from relu3_3 layer

        self.relu4_3 = nn.Sequential(*vgg[16:23]) # Extract features from relu4_3 layer


    def forward(self, x):

        out = {}

        x = self.relu1_2(x) # Forward pass through relu1_2

        out['relu1_2'] = x

        x = self.relu2_2(x) # Forward pass through relu2_2

        out['relu2_2'] = x

        x = self.relu3_3(x) # Forward pass through relu3_3

        out['relu3_3'] = x

        x = self.relu4_3(x) # Forward pass through relu4_3

        out['relu4_3'] = x

        return out
```

2. Residual Blocks:

Residual blocks consist of two convolutional layers with skip connections. These connections allow the network to learn residual mappings, making it easier to train deeper networks by avoiding vanishing gradients. They help preserve image details while enhancing the network's capacity to learn complex transformations.

```
class ResidualUnit(nn.Module):  
  
    def __init__(self, channels):  
        super(ResidualUnit, self).__init__()  
  
        kernel_size = 3  
  
        stride_size = 1  
  
        self.conv1 = Conv2dLayer(channels, channels, kernel_size=kernel_size,  
stride=stride_size) # First convolution  
  
        self.in1 = nn.InstanceNorm2d(channels, affine=True) # Normalize after the  
first conv  
  
        self.conv2 = Conv2dLayer(channels, channels, kernel_size=kernel_size,  
stride=stride_size) # Second convolution  
  
        self.in2 = nn.InstanceNorm2d(channels, affine=True) # Normalize after the  
second conv  
  
  
    def forward(self, x):  
  
        residual = x # Store the input for the residual connection  
  
        out = F.relu(self.in1(self.conv1(x))) # Apply the first conv and ReLU activation  
  
        out = self.in2(self.conv2(out)) # Apply the second conv  
  
        return out + residual # Add the residual to the output (skip connection)
```

AdaIN

1. VGG-19 Encoder (VGGEncoder):

The VGG-19 Encoder extracts deep features from content and style images using a pre-trained VGG-19 network, up to the relu4_1 layer. The encoder layers are frozen to prevent backpropagation, ensuring only the decoder and AdaIN layers are trained during optimization.

```
class VGGEncoder(nn.Module):

    def __init__(self):

        super(VGGEncoder, self).__init__()

        vgg = models.vgg19(weights=models.VGG19_Weights.DEFAULT).features # Load
pre-trained VGG-19 model

        self.encoder_layers = nn.Sequential()

        for i in range(21): # Include layers up to relu4_1

            self.encoder_layers.add_module(str(i), vgg[i])

        for param in self.encoder_layers.parameters():

            param.requires_grad = False


    def forward(self, x, get_style_features=False):

        if not get_style_features:

            return self.encoder_layers(x)


        features = [] # Initialize list to store style features

        for i, layer in enumerate(self.encoder_layers):

            x = layer(x) # Pass input through each layer

            if i in [1, 6, 11, 20]:

                features.append(x)

        return features
```

2. Decoder:

The Decoder generates the stylized image from features processed by AdaIN. It uses upsampling and convolutional layers to increase resolution and reduce channels, reconstructing the image in the reference style.

```
class Decoder(nn.Module):  
    def __init__(self):  
        super(Decoder, self).__init__()  
        # First set of layers to reduce the number of channels  
        self.decoder_layers = nn.Sequential(  
            nn.ReflectionPad2d((1, 1, 1, 1)), # Padding before conv layer to preserve  
dimensions  
            nn.Conv2d(512, 256, (3, 3)), # Conv layer to reduce channels  
            nn.ReLU(), # ReLU activation  
            nn.Upsample(scale_factor=2, mode='nearest') # Upsample the image to  
double its size  
        )  
  
    def forward(self, x):  
        return self.decoder_layers(x)
```


Dumoulin et al

1. Conditional Instance Normalization (CIN):

This class normalizes input features using Instance Normalization and modulates the normalization process based on style codes, allowing the model to apply different styles to the image.

```
class ConditionalInstanceNormalization(nn.Module):

    def __init__(self, num_styles, num_channels):

        super(ConditionalInstanceNormalization, self).__init__()

        self.normalize = nn.InstanceNorm2d(num_channels, affine=False)

        self.offset = nn.Parameter(0.01 * torch.randn(1, num_styles, num_channels))

        self.scale = nn.Parameter(1 + 0.01 * torch.randn(1, num_styles, num_channels))

    def forward(self, x, style_codes):

        b, c, h, w = x.size()

        x = self.normalize(x)

        gamma = torch.sum(self.scale * style_codes, dim=1).view(b, c, 1, 1)

        beta = torch.sum(self.offset * style_codes, dim=1).view(b, c, 1, 1)

        x = x * gamma + beta

        return x.view(b, c, h, w)
```

2. Image Style Transfer Network:

This class defines the neural network for the style transfer process. It applies convolutions, residual blocks, and upsampling layers to generate a stylized image based on the content image and style codes, using Conditional Instance Normalization to apply the style.

```
class ImageStyleTransferNet(nn.Module):

    def __init__(self, num_styles=16):
        super(ImageStyleTransferNet, self).__init__()

        self.conv1 = ConvolutionWithCIN(num_styles, 3, 32, 1, 'relu', 9)
        self.conv2 = ConvolutionWithCIN(num_styles, 32, 64, 2, 'relu', 3)
        self.conv3 = ConvolutionWithCIN(num_styles, 64, 128, 2, 'relu', 3)
        self.residual1 = CustomResidualBlock(num_styles, 128, 128)
        self.upsampling1 = CustomUpsampleBlock(num_styles, 128, 64)
        self.conv4 = ConvolutionWithCIN(num_styles, 32, 3, 1, 'linear', 9)

    def forward(self, x, style_codes):
        x = self.conv1(x, style_codes)
        x = self.conv2(x, style_codes)
        x = self.conv3(x, style_codes)
        x = self.residual1(x, style_codes)
        x = self.upsampling1(x, style_codes)
        x = self.conv4(x, style_codes)

        return x
```

Dumoulin v2

1. ADAIN for Style Modulation:

This part enables dynamic modulation of the content image with the style using the mean and standard deviation from the style image. Unlike Dumoulin's original model, style codes are removed, and dynamic style modulation via ADAIN allows transferring any unseen style.

```
class AdaptiveInstanceNormalization(nn.Module):  
  
    def __init__(self):  
        super(AdaptiveInstanceNormalization, self).__init__()  
        self.normalize = nn.InstanceNorm2d(num_channels, affine=False)  
  
    def forward(self, x, style_statistics):  
        b, c, h, w = x.size()  
        x = self.normalize(x)  
        gamma = style_statistics['mean'] # Apply extracted style statistics  
        dynamically  
        beta = style_statistics['std'] # Apply extracted style statistics dynamically  
        x = x * gamma + beta # Modulate the style using the statistics  
        return x
```

Dumoulin V2 – multi-style

1. Image StyleTransfer Net

The forward method splits the content image into two halves, applying Style 1 to the left and Style 2 to the right using CIN. A blending region smooths the transition, and the parts are concatenated to form the final stylized image.

```
class ImageStyleTransferNet(nn.Module):

    ...

    def forward(self, content_image, style_image1, style_image2):

        B, C, H, W = content_image.shape

        half_W = W // 2

        content_left = content_image[:, :, :, :half_W]

        content_right = content_image[:, :, :, half_W:]

        stats1 = self.style_extractor.extract_statistics(style_image1)

        cin_params1 = self.param_generator(stats1)

        stats2 = self.style_extractor.extract_statistics(style_image2)

        cin_params2 = self.param_generator(stats2)

        left_out = forward_pass(content_left, cin_params1)

        right_out = forward_pass(content_right, cin_params2)

        blend_start_global = half_W - overlap

        blend_left = left_out[:, :, :, blend_start_global:]

        blend_right = right_out[:, :, :, :blend_width]

        mask = torch.linspace(1.0, 0.0, blend_width, device=content_image.device)

        blended = mask * blend_left + (1 - mask) * blend_right

        output = torch.cat([left_nonblend, blended, right_nonblend], dim=3)

        return output
```

Dumoulin V2 – video style transfer

1. Temporal consistency:

This feature using optical flow. This ensures smooth transitions between consecutive frames in the video, avoiding abrupt changes or flickering.

```
# Compute optical flow from prev_resized to current resized frame
curr_resized = resized_frames[i]

prev_gray = cv2.cvtColor(prev_resized, cv2.COLOR_RGB2GRAY)
curr_gray = cv2.cvtColor(curr_resized, cv2.COLOR_RGB2GRAY)
flow = cv2.calcOpticalFlowFarneback(prev_gray, curr_gray, None, 0.5, 3, 15, 3, 5, 1.2, 0)

# Create flow map
h, w = flow.shape[:2]
y, x = np.mgrid[0:h, 0:w]
flow_map = np.stack((x, y), axis=-1).astype(np.float32)
flow_map += flow

flow_map[..., 0] = 2.0 * flow_map[..., 0] / max(w - 1, 1) - 1.0
flow_map[..., 1] = 2.0 * flow_map[..., 1] / max(h - 1, 1) - 1.0

# Convert to torch [1, H, W, 2]
flow_map = torch.from_numpy(flow_map).unsqueeze(0).to(device).float()

# Warp previous stylized frame
prev_stylized_warped = F.grid_sample(prev_stylized, flow_map, mode='bilinear',
padding_mode='border', align_corners=False)

# Blend: 80% new stylized + 20% warped previous for smoothness
stylized = 0.8 * stylized + 0.2 * prev_stylized_warped
```

4.5 Visual representation of the results

I will be showing the final results of each experiment for each model and I will show the learning progress of the best hyperparameter chosen through the surveys. I will talk more about it in the evaluation section. More outputs could be found in the portfolio folder on GitHub repo or you can generate your own through the website I attached.

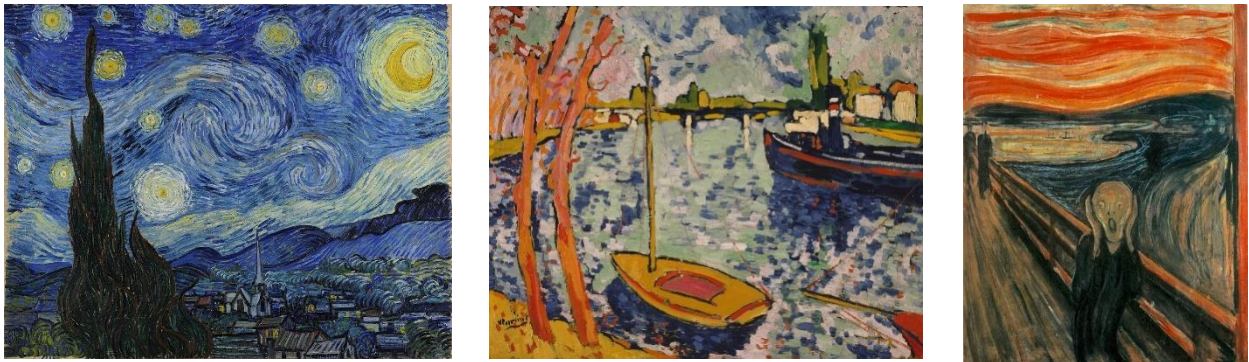
Here are some images for reference in 4.5:

Content images that will be used in the experiment and visuals shown later:



Figures (2) Content images used in the visuals

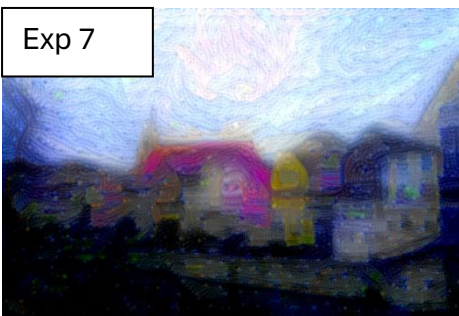
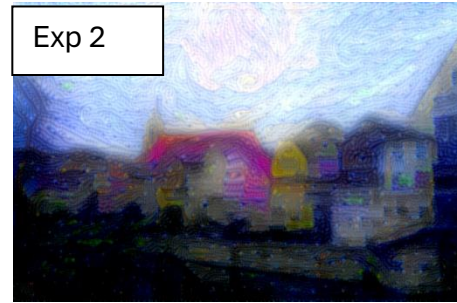
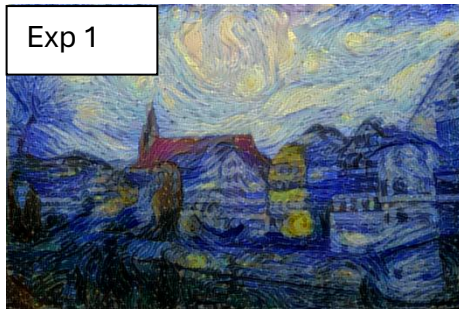
Some of the style images that will be used in the experiment and visuals shown later:



Figures (3) Some style images used in the visuals

1. gatys et al visual results

1.1. Hyperparameter exploration results:



Figures (4) the final results of the gaty's 8 hyperparameter experiments

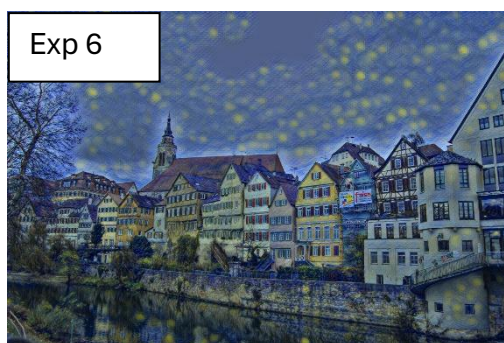
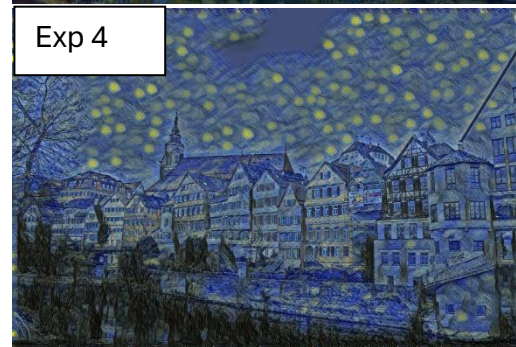
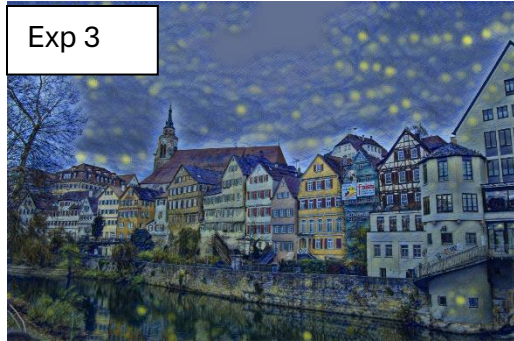
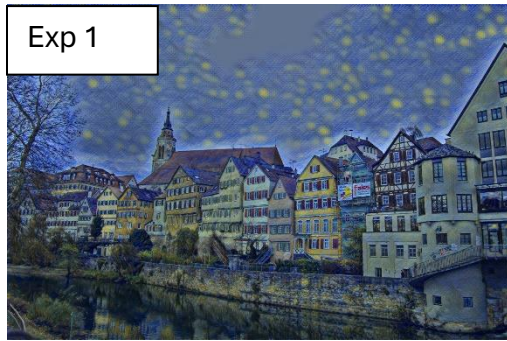
1.2. Learning progress of Gaty's experiment one through the iterations:



Figures (5) gaty's experiment one learning progress in each 50 iterations

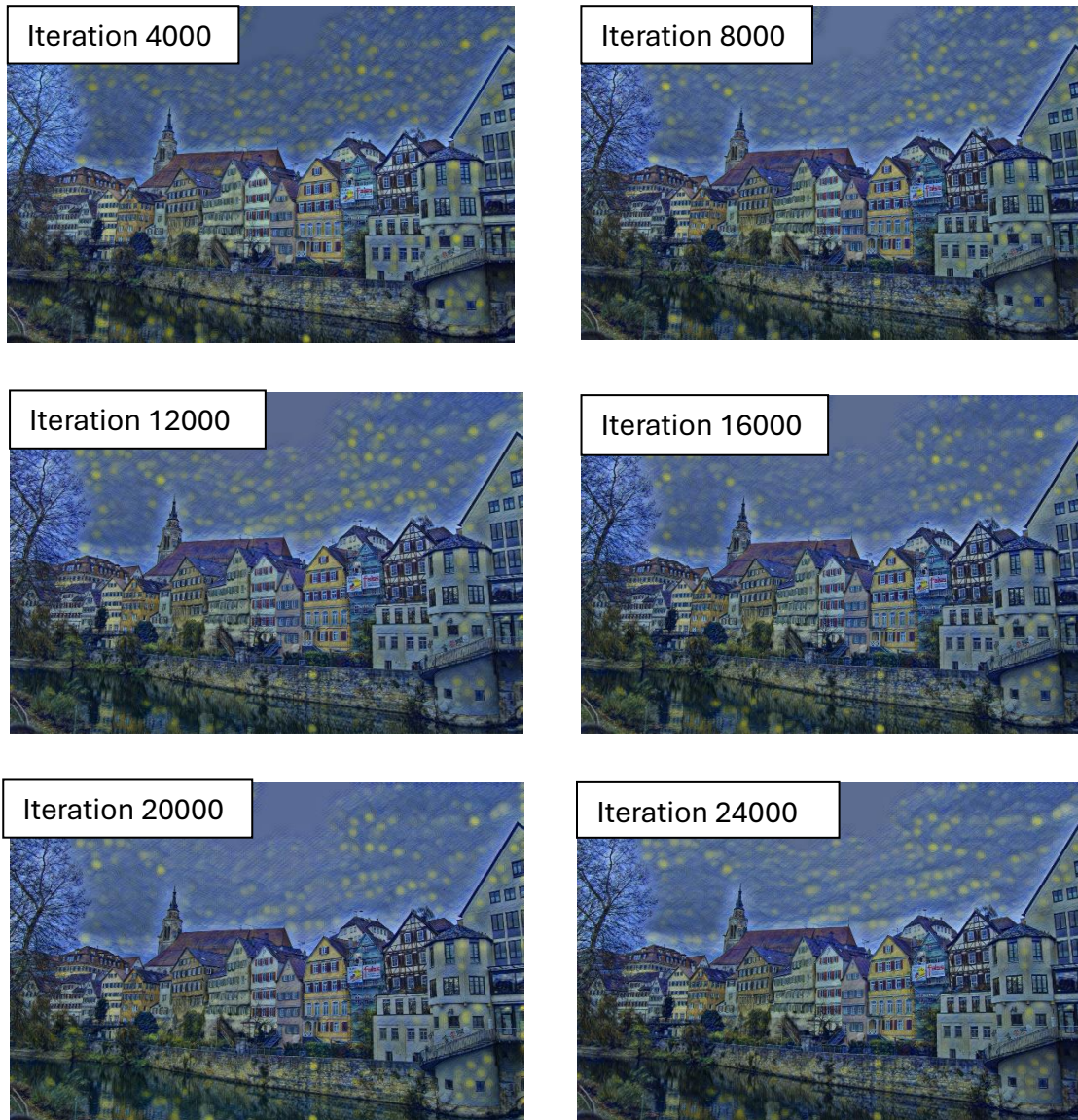
2. Johnson et al visual results

2.1. Hyperparameter exploration results:



Figures (6) the final results of Johnson's 8 hyperparameter experiments

2.2. Learning progress of Johnson's experiment six through the iterations:



Figures (7) Jonson's experiment one learning progress in each 4000 iterations

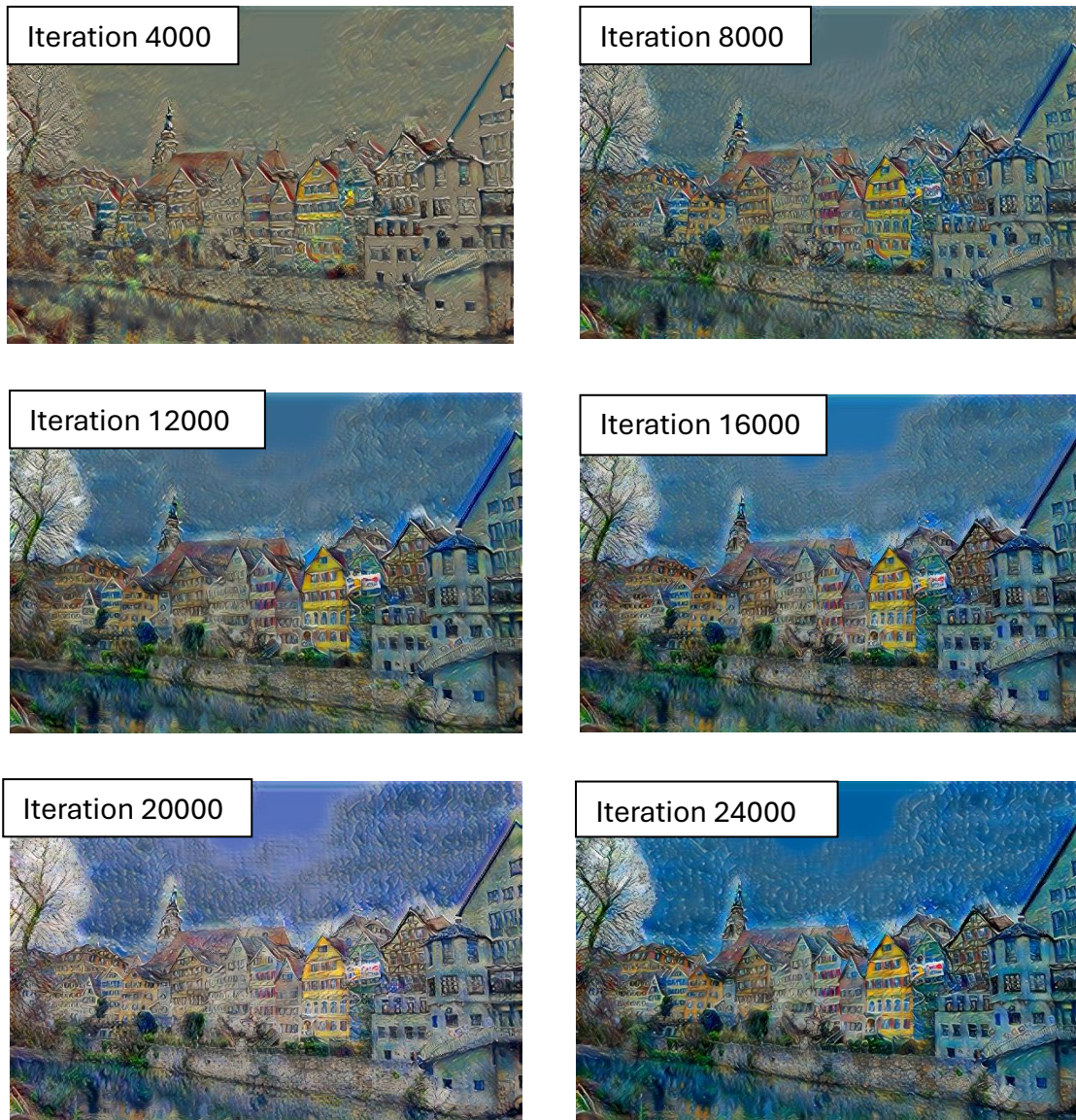
3. AdaIN et al visual results:

3.1. Hyperparameter exploration results:



Figures (8) the final results of AdaIN's 7 hyperparameter experiments, exp 5 isn't included as it gave a blank photo

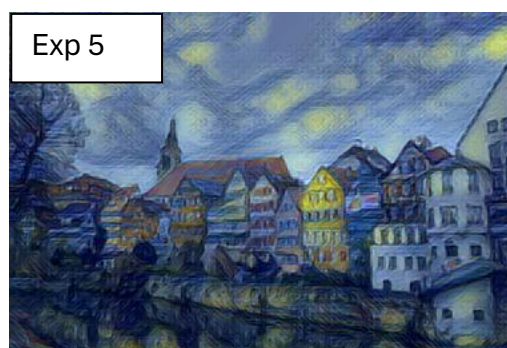
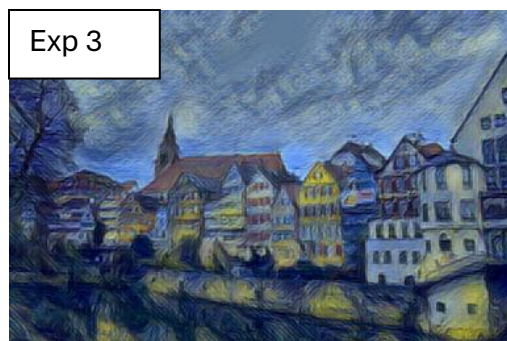
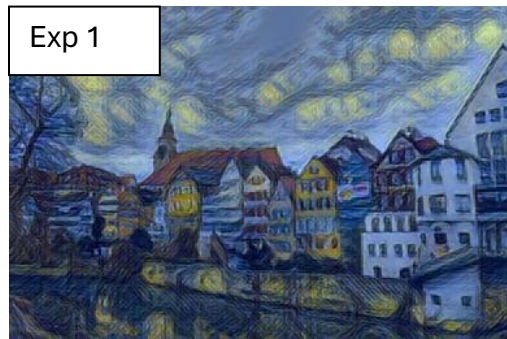
3.2. Learning progress of Johnson's experiment one through the iterations:



Figures (9) Jonson's experiment one learning progress in each 4000 iterations

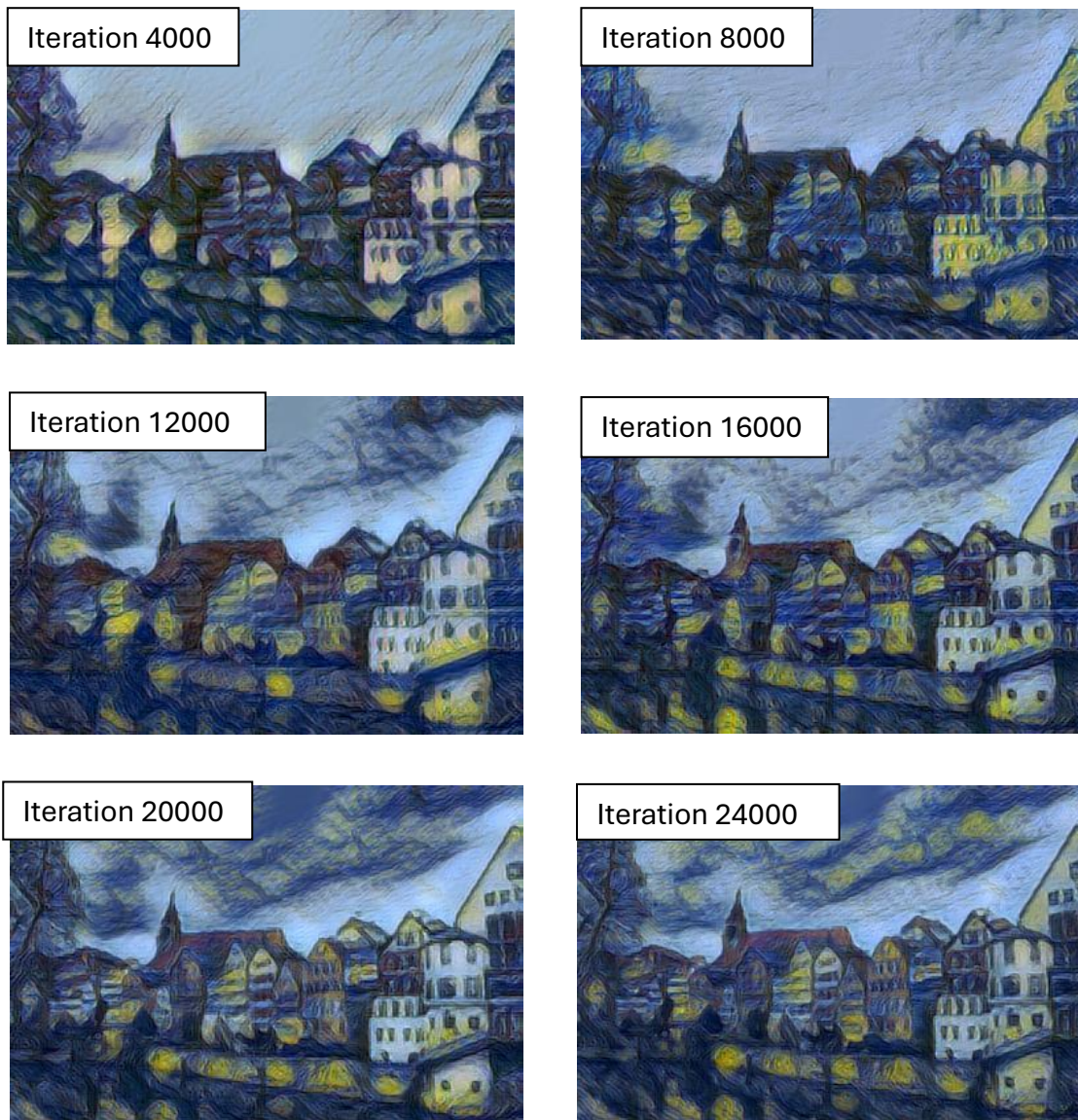
4. Dumoulin et al visual results:

4.1. Hyperparameter exploration results:



Figures (10) the final results of Dumoulin's 8 hyperparameter experiments

4.2. Learning progress of Johnson's experiment seven through the iterations:

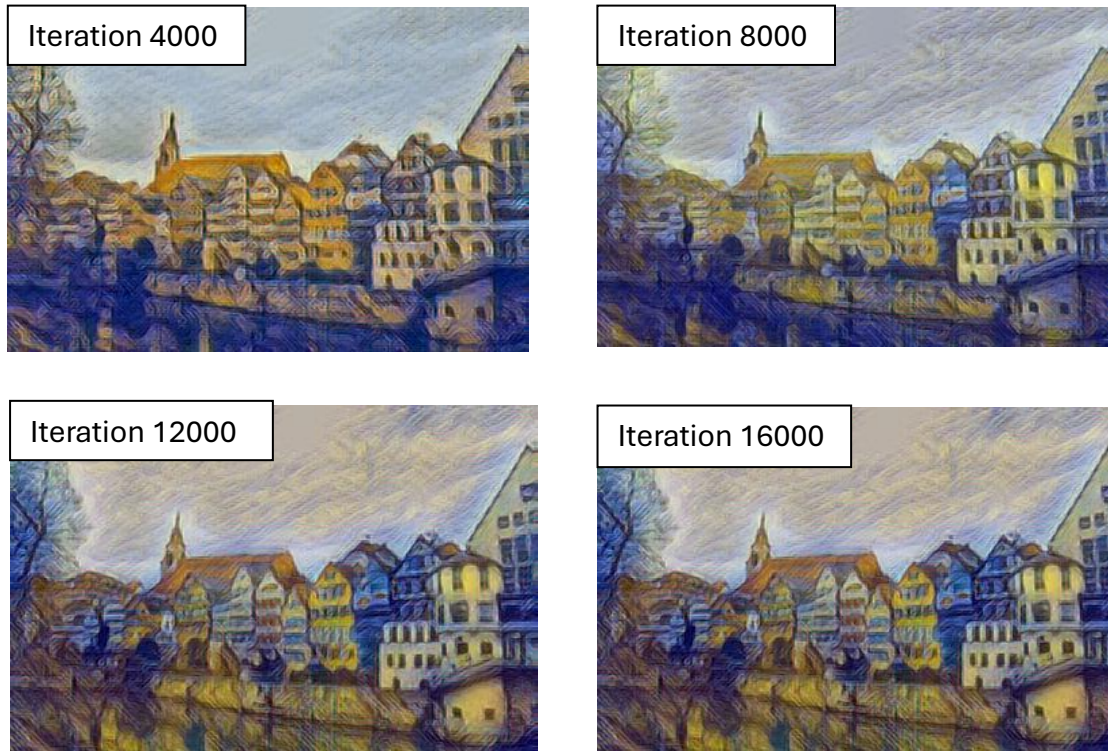


Figures (11) Dumoulin's experiment seven learning progress in each 4000 iterations

5. Dumoulin V2

I didn't explore hyperparameters as it was close to Dumoulin et al paper implementation, so I used the same best parameter.

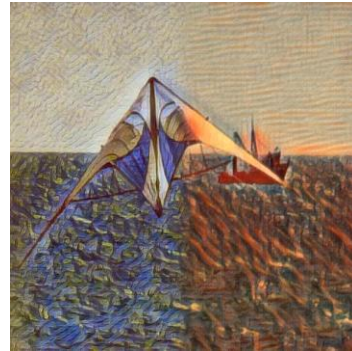
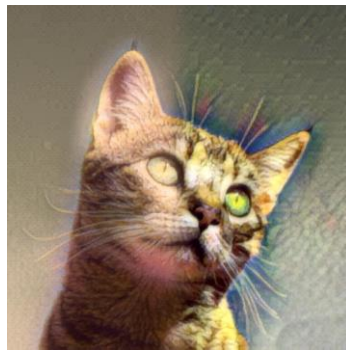
5.1. Learning progress of Dumoulin's V2 through the iterations:



Figures (12) Dumoulin's V2 learning progress in each 4000 iterations

6. Dumoulin's V2 multi-style:

6.1. I didn't explore hyperparameters as it was close to Dumoulin et al paper implementation, so I used the same best parameter.



Figures (13) Some of Dumoulin's V2 multi-style outputs generated

7. Dumoulin's V2 video style transfer:

I can't attach the video outputs but you can find them in the portfolio folder in the repo on GitHub or you can generate your own stylized video using the website.

5. Evaluation (word count: 2145)

In this section, I present the evaluation carried out on the developed Neural Style Transfer (NST) models and the associated website. The evaluation is divided into two parts: qualitative and quantitative.

The qualitative evaluation focuses on feedback obtained from user surveys. The first set of surveys was conducted to evaluate the hyperparameter settings for four NST models: AdaIN, Johnson, Gatys, and Dumoulin. The aim was to understand user preferences regarding the hyperparameters used in these models, and determine which model best aligned with their expectations for style transfer performance. The fifth survey evaluated the usability and user experience of the Neural Style Transfer website, seeking insights into its effectiveness, ease of use, and overall satisfaction.

The quantitative evaluation measures the technical performance of the NST models. It includes the analysis of content and style loss MSE graphs for each model, processing time evaluations, as well as assessments based on perceptual similarity metrics such as SSIM (Structural Similarity Index), DF (Dissimilarity Factor), SF (Style Fidelity), and LPIPS (Learned Perceptual Image Patch Similarity).

5.1. Qualitative Evaluation

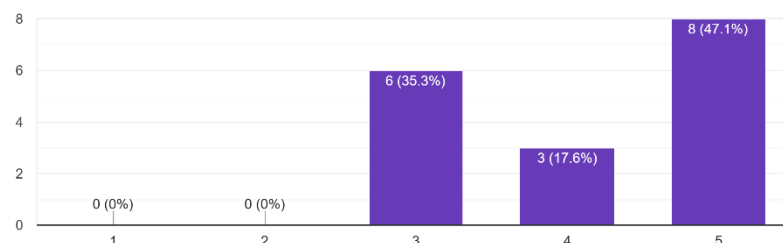
User surveys:

1. Johnson survey

A total of 17 responses were gathered to evaluate 8 experiments, each testing different combinations of content weight, style weight, and TV weight to explore their effects on the Johnson model's output.

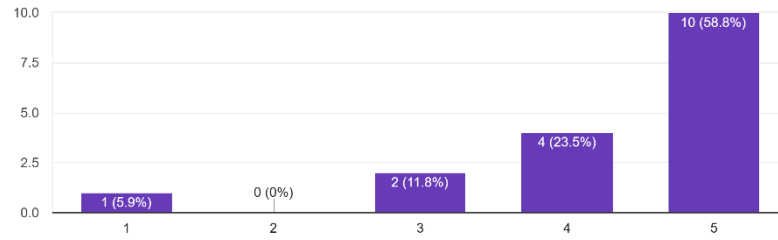
Experiment 1 (Content Weight: 1.0, Style Weight: 1e5, TV Weight: 1e-6) aimed for a balance between content preservation and style transfer. Results showed a good balance, with a slight emphasis on style. Most participants were satisfied with the content preservation.

How would you rate the stylization quality of this image?
17 responses



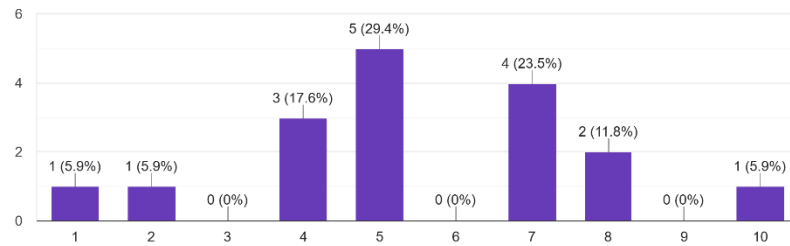
How well does this image keep the original content from the content image?

17 responses



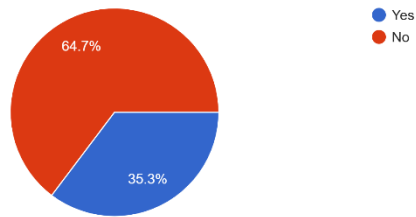
How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content

17 responses



Did you see any problems or strange parts in the image?

17 responses



How pleasing does this image look to you?

17 responses

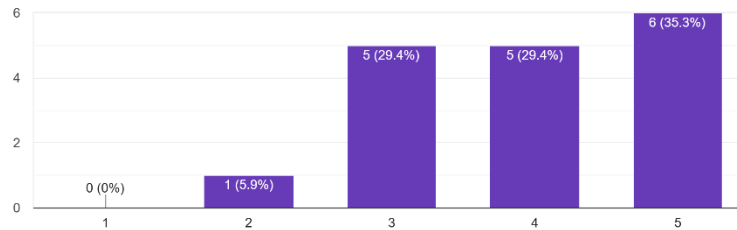
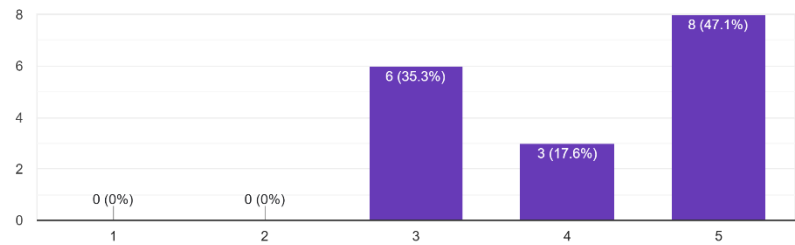


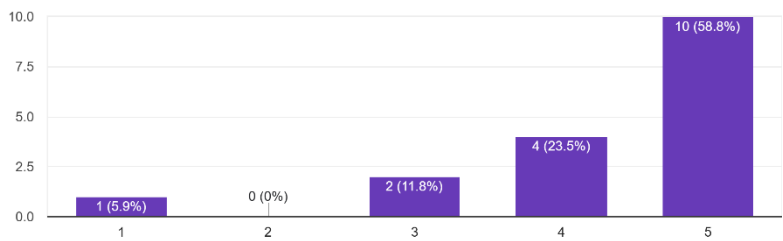
Figure (14) first, second, third, fourth, fifth survey questions and their responses for Johnson's experiment 1

Experiment 2 (Content Weight: 5.0, Style Weight: 1e5, TV Weight: 1e-6) focused more on content preservation. Participants appreciated the improved content balance, but noted a slightly weaker style compared to Experiment 1.

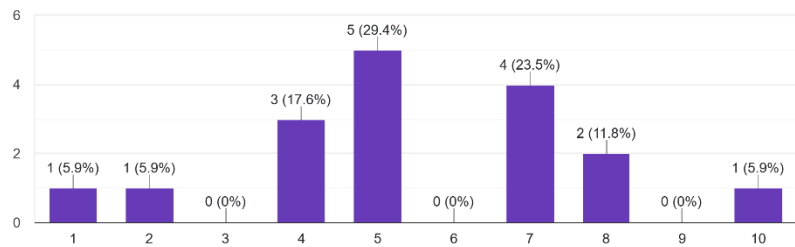
How would you rate the stylization quality of this image?
17 responses



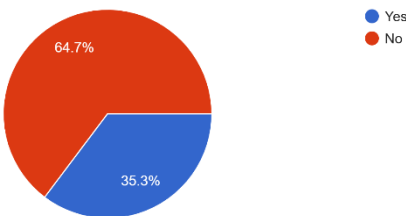
How well does this image keep the original content from the content image?
17 responses



How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content
17 responses



Did you see any problems or strange parts in the image?
17 responses



How pleasing does this image look to you?
17 responses

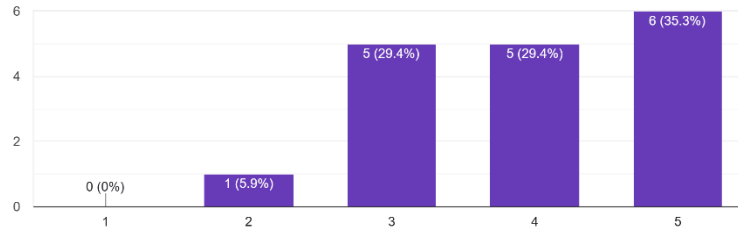
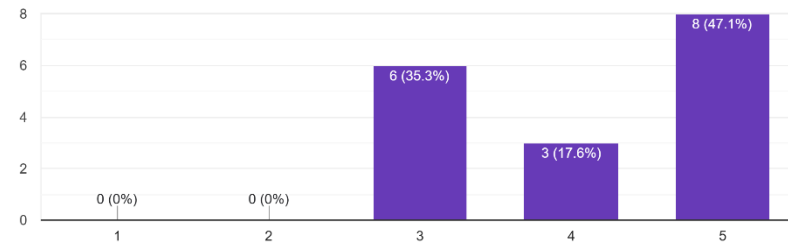


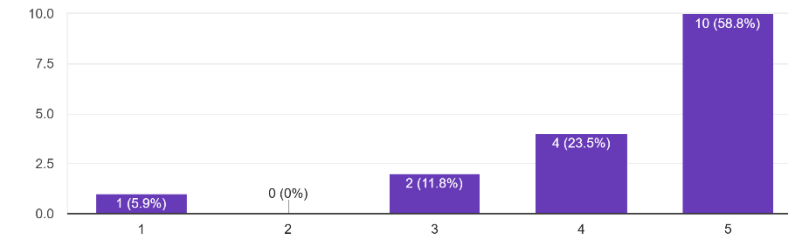
Figure (15) first, second, third, fourth, fifth survey questions and their responses for Johnson's experiment 2

Experiment 3 (Content Weight: 0.5, Style Weight: $5e4$, TV Weight: $5e-6$) aimed for a more balanced output with smoother results due to the higher TV loss. Participants liked the smoother transition but felt the style was less intense.

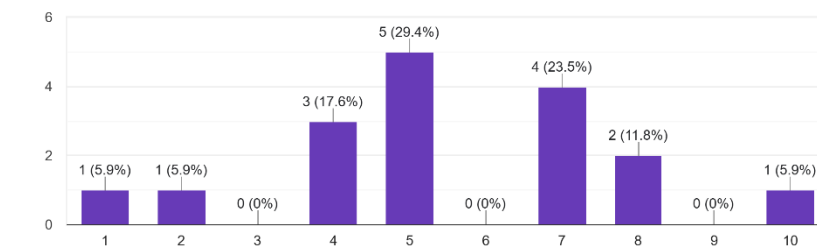
How would you rate the stylization quality of this image?
17 responses



How well does this image keep the original content from the content image?
17 responses

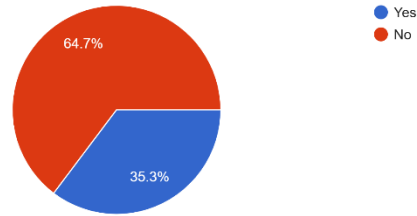


How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content
17 responses



Did you see any problems or strange parts in the image?

17 responses



How pleasing does this image look to you?

17 responses

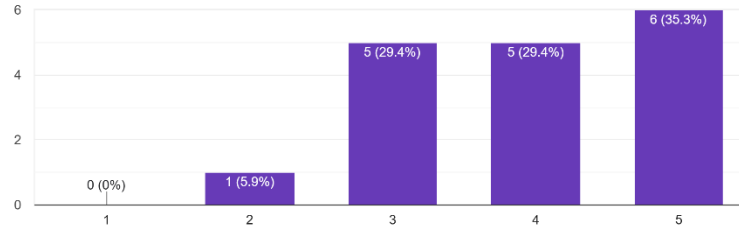
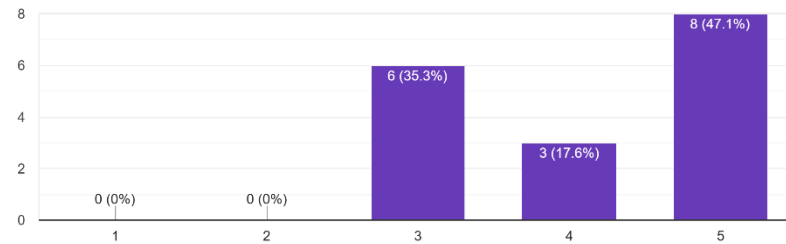


Figure (16) first, second, third, fourth, fifth survey questions and their responses for Johnson's experiment 3

Experiment 4 (Content Weight: 0.1, Style Weight: 1e5, TV Weight: 1e-6) gave almost complete dominance to style. The resulting images were highly stylized, with less content preservation, but most participants were satisfied with the vivid style.

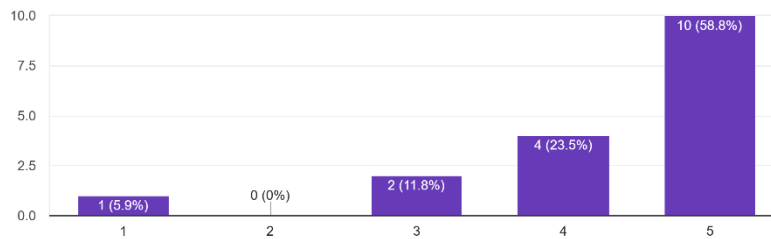
How would you rate the stylization quality of this image?

17 responses



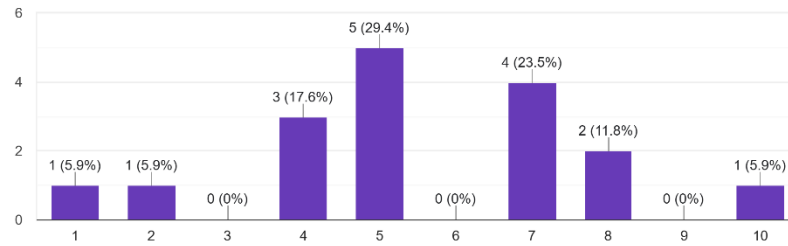
How well does this image keep the original content from the content image?

17 responses



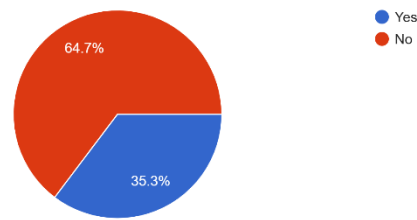
How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content

17 responses



Did you see any problems or strange parts in the image?

17 responses



How pleasing does this image look to you?

17 responses

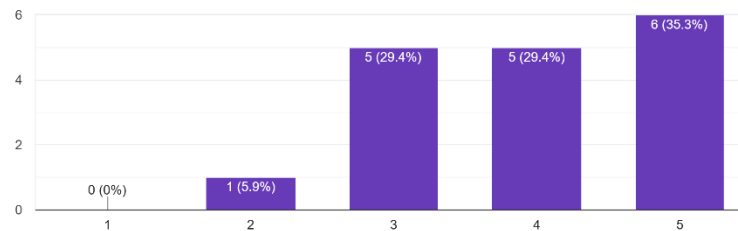
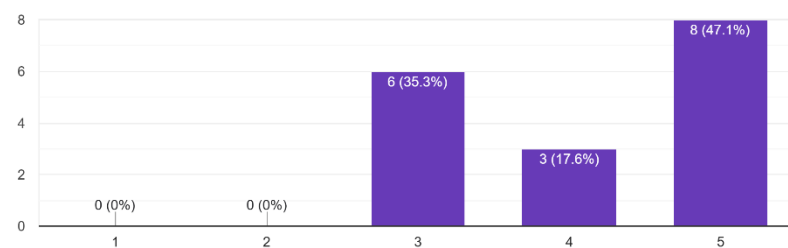


Figure (17) first, second, third, fourth, fifth survey questions and their responses for Johnson's experiment 4

Experiment 5 (Content Weight: 1.0, Style Weight: 1e4, TV Weight: 1e-6) reduced the style weight slightly while keeping content balanced. The content was more pronounced, but some participants felt the style was less bold.

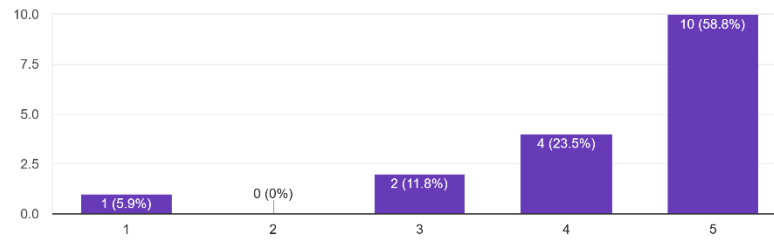
How would you rate the stylization quality of this image?

17 responses



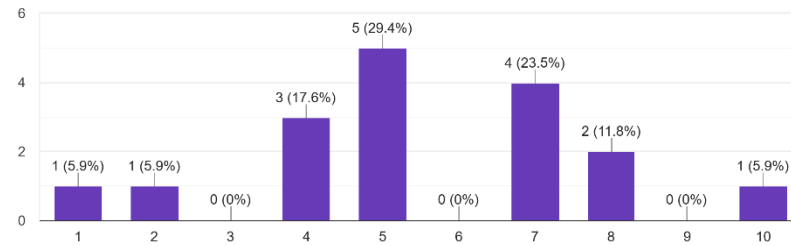
How well does this image keep the original content from the content image?

17 responses



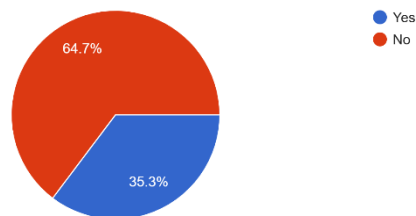
How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content

17 responses



Did you see any problems or strange parts in the image?

17 responses



How pleasing does this image look to you?

17 responses

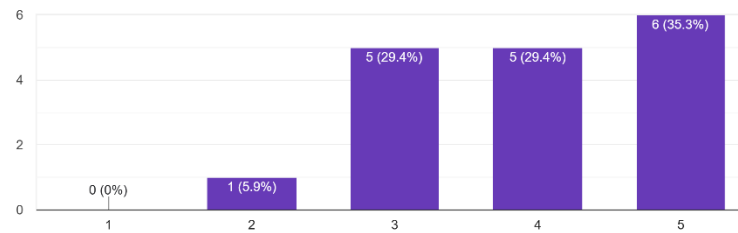
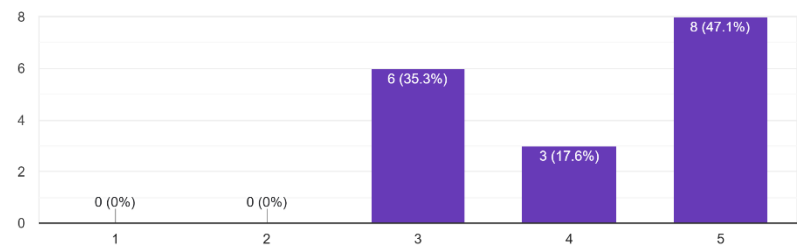


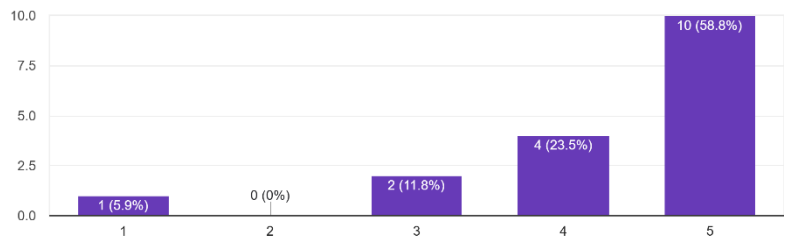
Figure (18) first, second, third, fourth, fifth survey questions and their responses for Johnson's experiment 5

Experiment 6 (Content Weight: 1.0, Style Weight: 1e5, TV Weight: 1e-5) tested the effect of increased TV loss for smoothing. The results were well-received, with smoother outputs, though some felt the style was slightly less impactful.

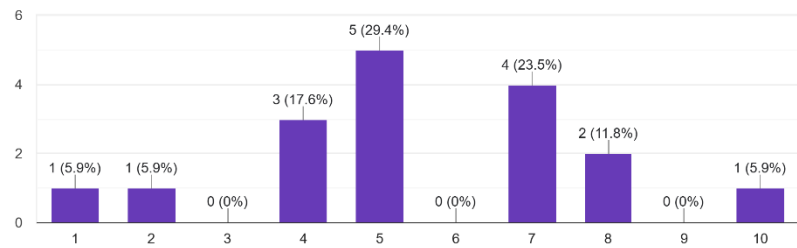
How would you rate the stylization quality of this image?
17 responses



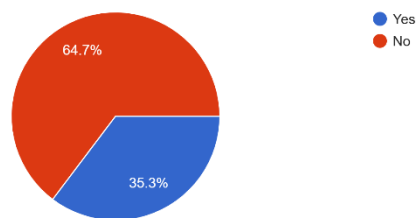
How well does this image keep the original content from the content image?
17 responses



How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content
17 responses



Did you see any problems or strange parts in the image?
17 responses



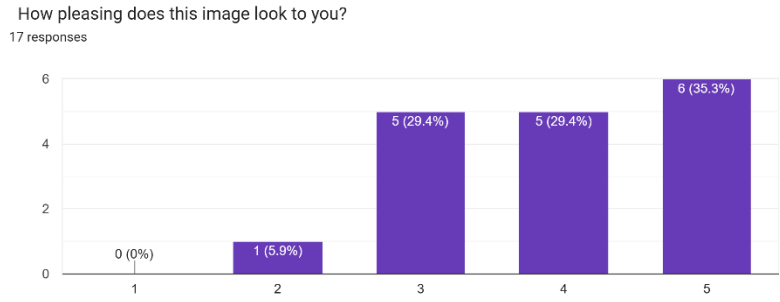
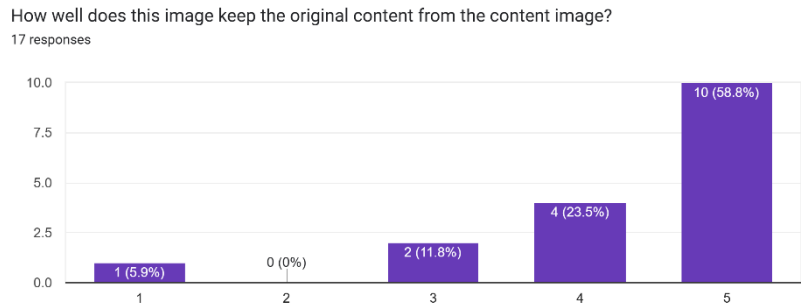
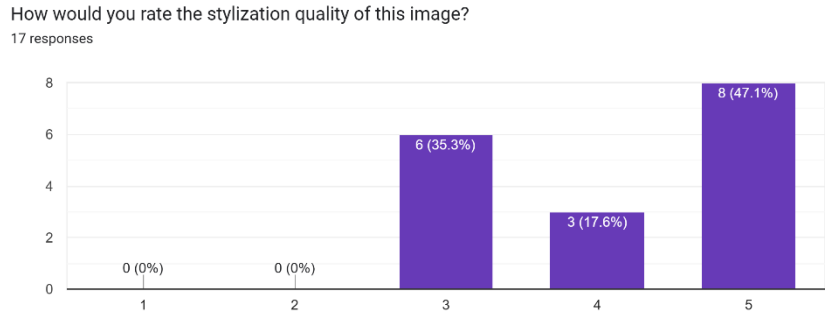


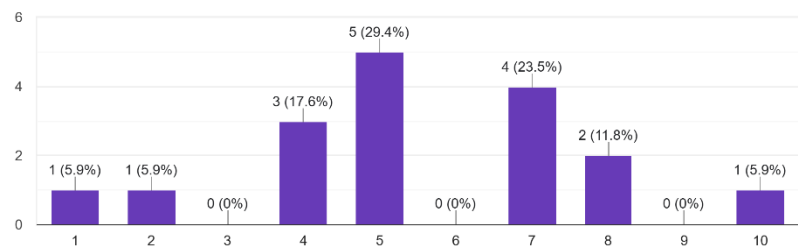
Figure (17) first, second, third, fourth, fifth survey questions and their responses for Johnson's experiment 6

Experiment 7 (Content Weight: 1.0, Style Weight: 1e5, TV Weight: 1e-7) had minimal noise reduction, preserving more fine details. Feedback was mixed, with some appreciating the clearer content and others noticing more visible noise.

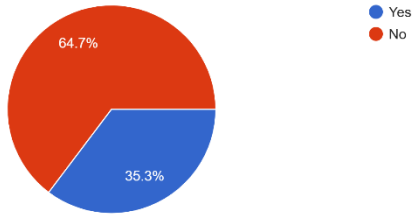


How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content

17 responses



Did you see any problems or strange parts in the image?
17 responses



How pleasing does this image look to you?
17 responses

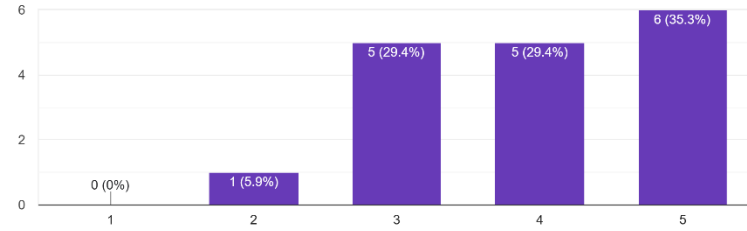
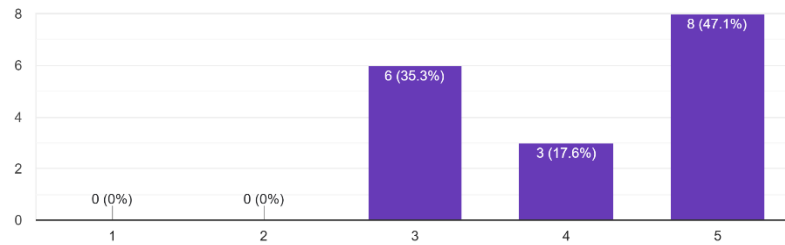


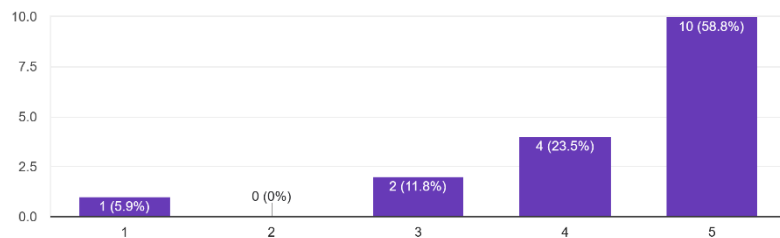
Figure (18) first, second, third, fourth, fifth survey questions and their responses for Johnson's experiment 7

Experiment 8 (Content Weight: 10.0, Style Weight: 1e6, TV Weight: 1e-5) tested extreme values for both content and style weights. Most participants liked the strong content preservation, but the style remained prominent.

How would you rate the stylization quality of this image?
17 responses

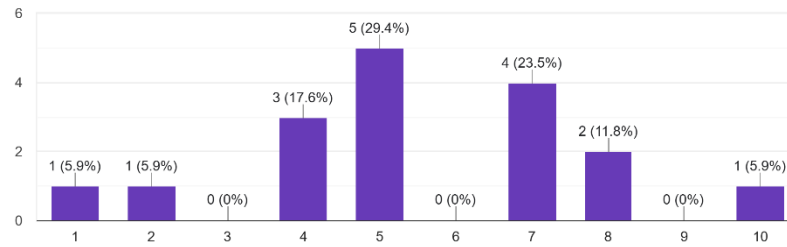


How well does this image keep the original content from the content image?
17 responses



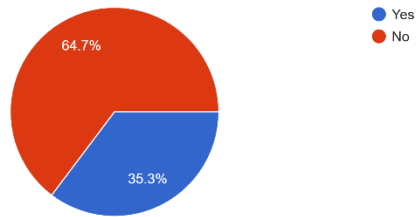
How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content

17 responses



Did you see any problems or strange parts in the image?

17 responses



How pleasing does this image look to you?

17 responses

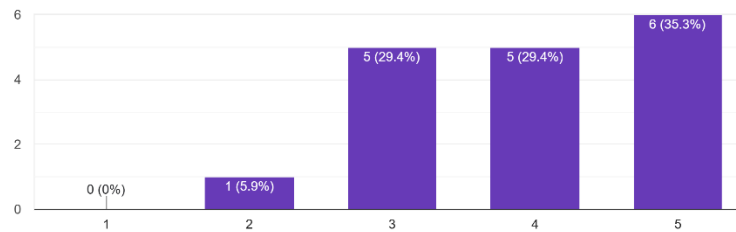


Figure (19) first, second, third, fourth, fifth survey questions and their responses for Johnson's experiment 8

Final choice question, which helped me choose the best parameters:

If you had to select only one image, which would it be?

17 responses

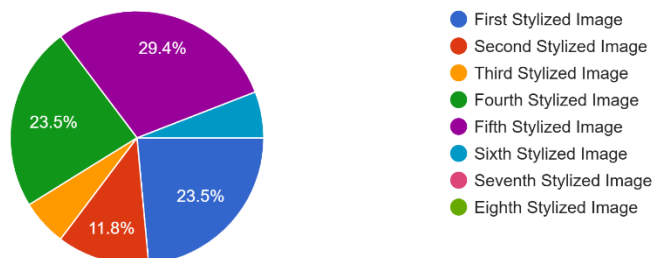


Figure (20) Final choice question for Johnson's experiments

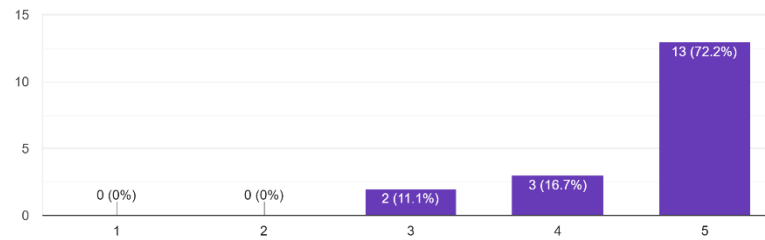
2. Dumoulin survey

A total of 18 responses were collected to evaluate the 8 Dumoulin experiments, each using different combinations of content weight, style weight, and TV weight.

Experiment 1 (Content Weight: 1.0, Style Weight: 10.0, TV Weight: 1e-6) focused on balancing content and style, with positive feedback on both content preservation and style transfer, though minimal noise reduction.

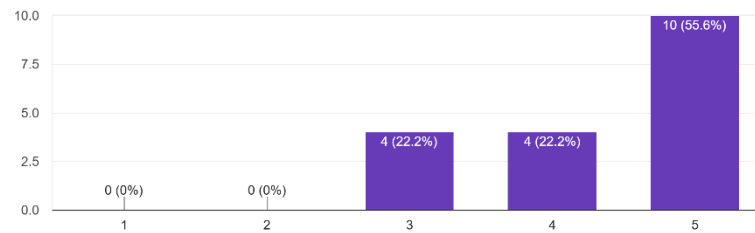
How would you rate the stylization quality of this image?

18 responses



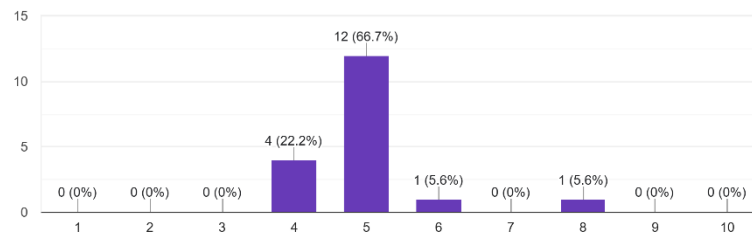
How well does this image keep the original content from the content image?

18 responses



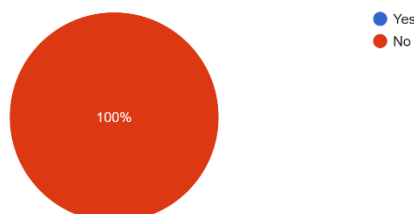
How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content

18 responses



Did you see any problems or strange parts in the image?

18 responses



How pleasing does this image look to you?

18 responses

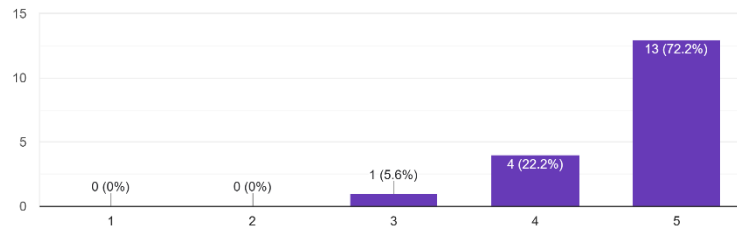
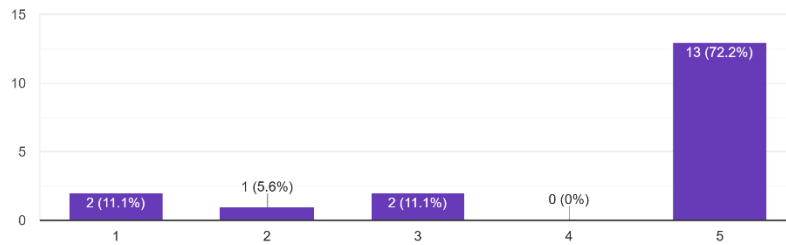


Figure (21) first, second, third, fourth, fifth survey questions and their responses for Dumoulin's experiment 1

Experiment 2 (Content Weight: 5.0, Style Weight: 5.0, TV Weight: $1e-6$) f was well-received for its balanced preservation of content and style, with minimal noise.

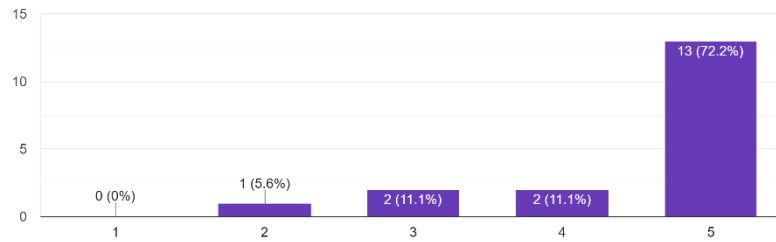
How would you rate the stylization quality of this image?

18 responses



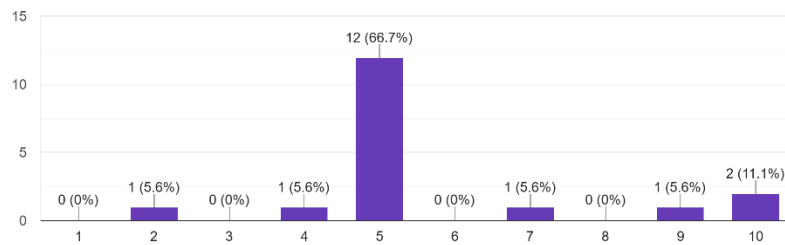
How well does this image keep the original content from the content image?

18 responses

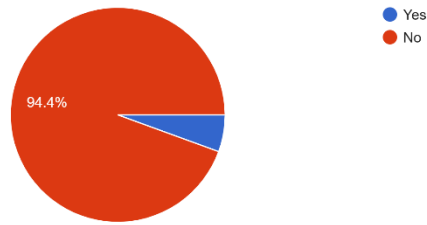


How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content

18 responses



Did you see any problems or strange parts in the image?
18 responses



How pleasing does this image look to you?
18 responses

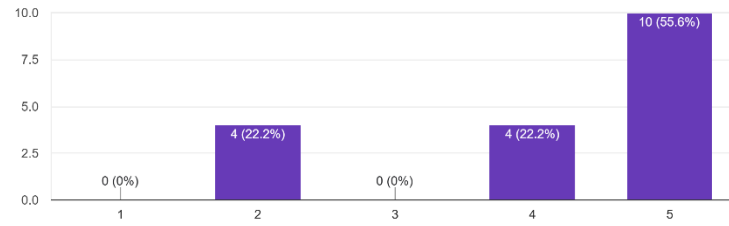
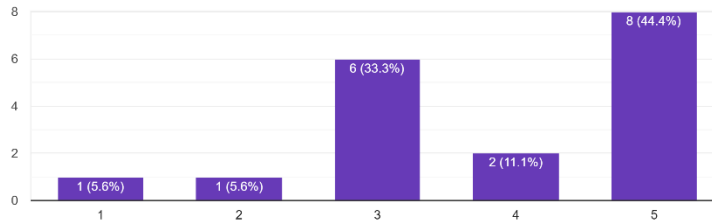


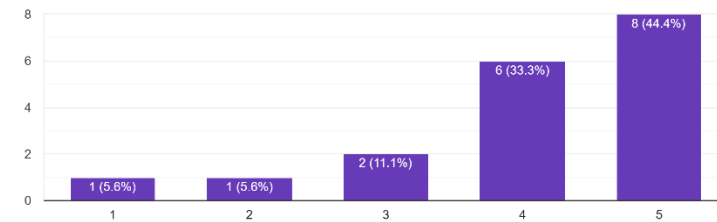
Figure (22) first, second, third, fourth, fifth survey questions and their responses for Dumoulin's experiment 2

Experiment 3 (Content Weight: 1.0, Style Weight: 10.0, TV Weight: 1e-3) added more smoothing, which participants appreciated, though some felt the style was slightly reduced.

How would you rate the stylization quality of this image?
18 responses

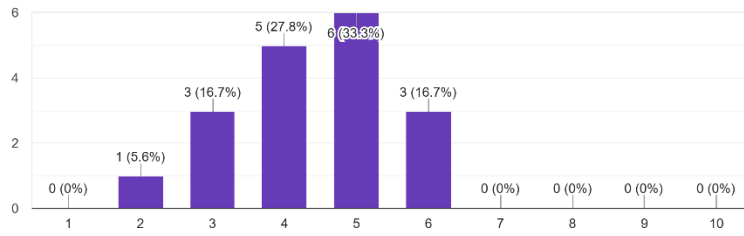


How well does this image keep the original content from the content image?
18 responses



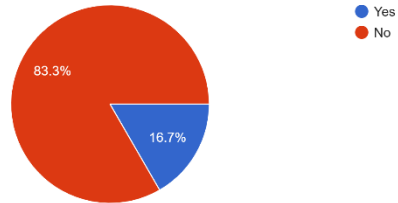
How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content

18 responses



Did you see any problems or strange parts in the image?

18 responses



How pleasing does this image look to you?

18 responses

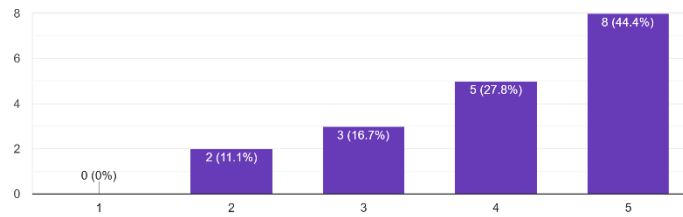
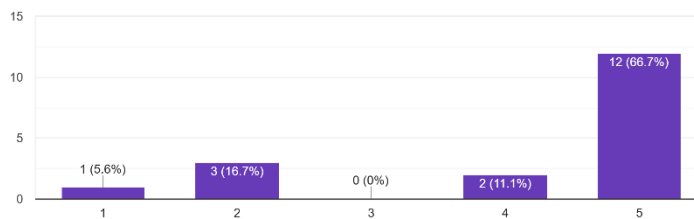


Figure (23) first, second, third, fourth, fifth survey questions and their responses for Dumoulin's experiment 3

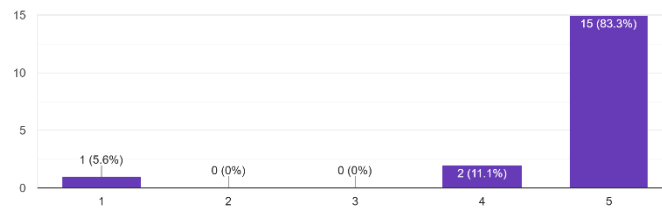
Experiment 4 (Content Weight: 5.0, Style Weight: 10.0, TV Weight: 1e-6) preserved content well but lacked in style intensity.

How would you rate the stylization quality of this image?

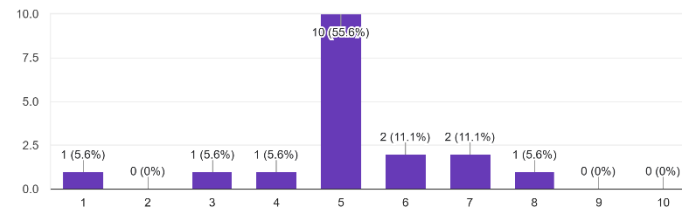
18 responses



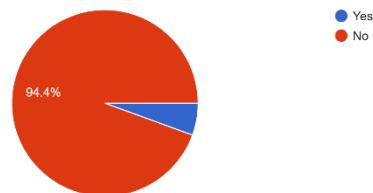
How well does this image keep the original content from the content image?
18 responses



How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content
18 responses



Did you see any problems or strange parts in the image?
18 responses



How pleasing does this image look to you?
18 responses

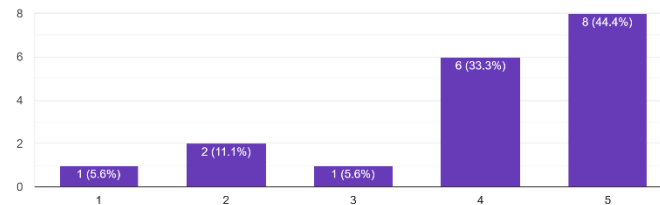
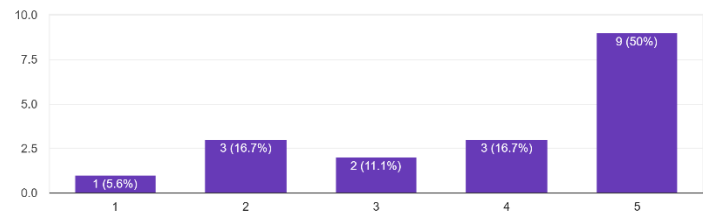


Figure (24) first, second, third, fourth, fifth survey questions and their responses for Dumoulin's experiment 4

Experiment 5 (Content Weight: 1.0, Style Weight: 10.0, TV Weight: 1e1) showed heavy smoothing, which some found appealing, though it led to a loss of fine details.

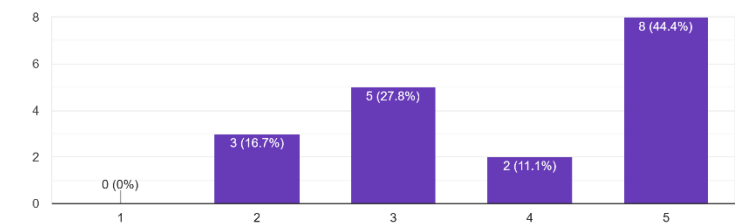
How would you rate the stylization quality of this image?

18 responses



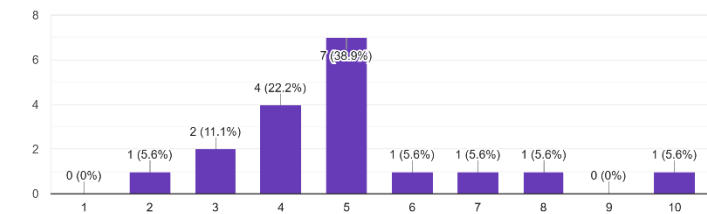
How well does this image keep the original content from the content image?

18 responses



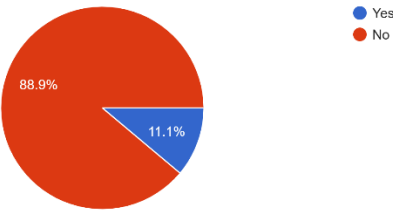
How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content

18 responses



Did you see any problems or strange parts in the image?

18 responses



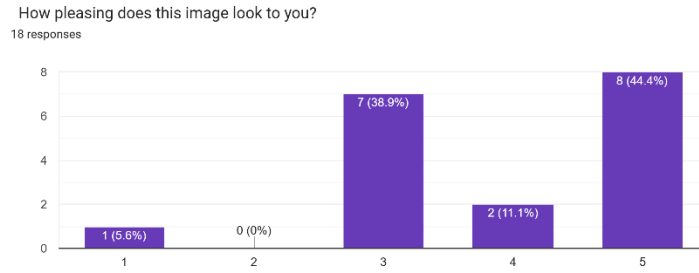
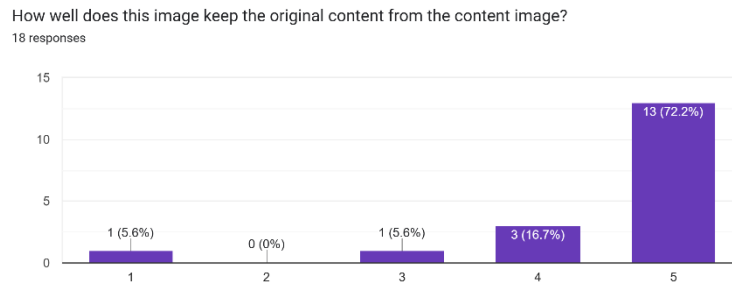
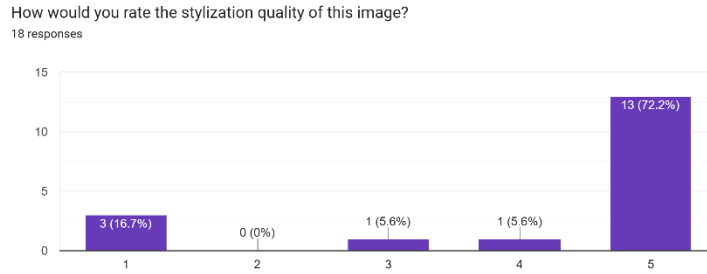


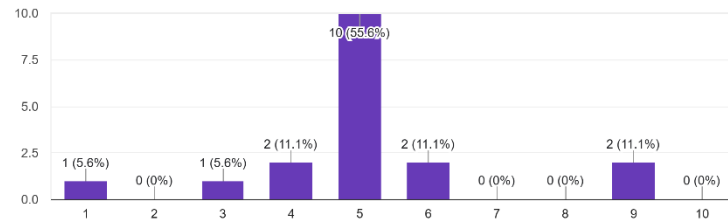
Figure (25) first, second, third, fourth, fifth survey questions and their responses for Dumoulin's experiment 5

Experiment 6 (Content Weight: 11.0, Style Weight: 10.0, TV Weight: 1e-6) prioritized content preservation with minimal smoothing, but the style could have been more prominent.

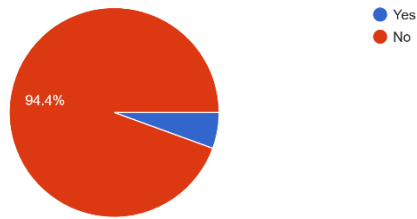


How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content

18 responses



Did you see any problems or strange parts in the image?
18 responses



How pleasing does this image look to you?
18 responses

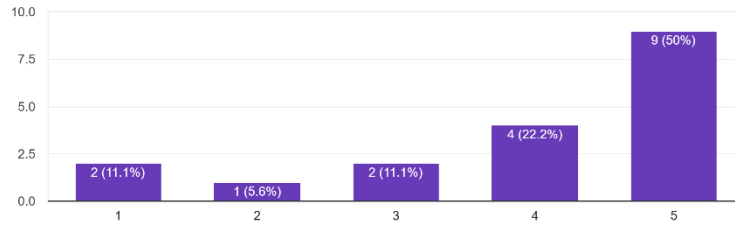
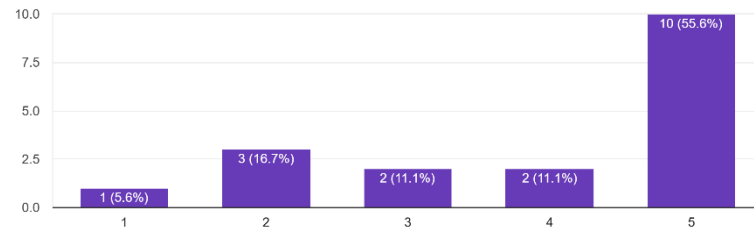


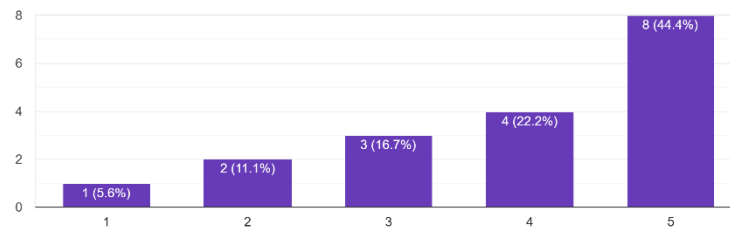
Figure (26) first, second, third, fourth, fifth survey questions and their responses for Dumoulin's experiment 6

Experiment 7 (Content Weight: 1.0, Style Weight: 15.0, TV Weight: 1e-3) emphasized style, but some content was lost in the process, though most found the result pleasing.

How would you rate the stylization quality of this image?
18 responses

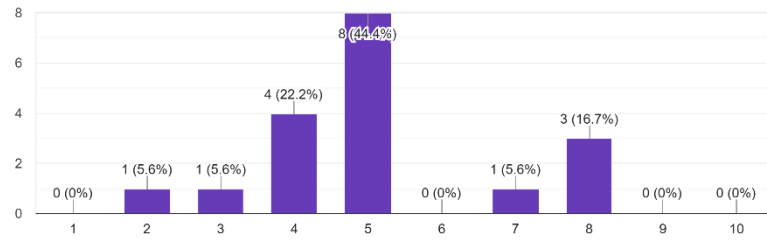


How well does this image keep the original content from the content image?
18 responses



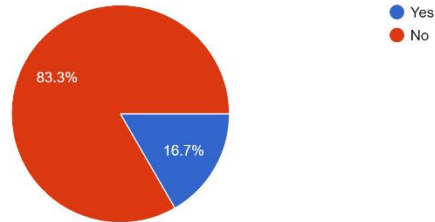
How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content

18 responses



Did you see any problems or strange parts in the image?

18 responses



How pleasing does this image look to you?

18 responses

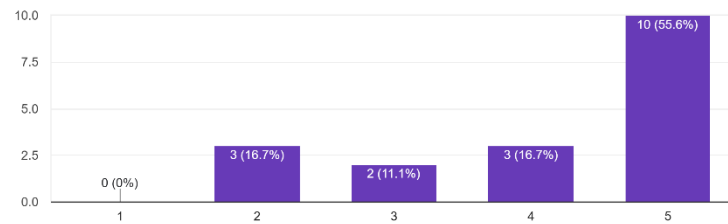
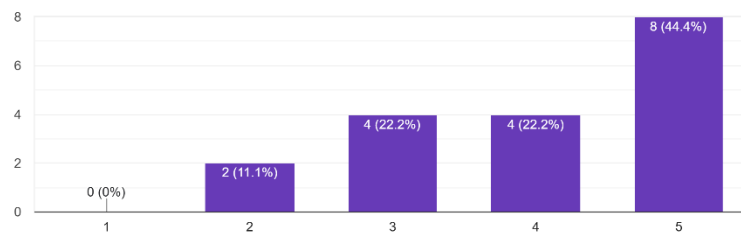


Figure (27) first, second, third, fourth, fifth survey questions and their responses for Dumoulin's experiment 7

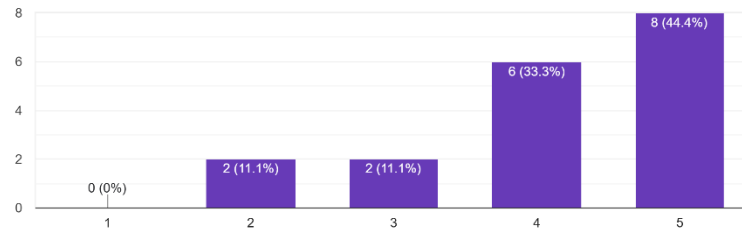
Experiment 8 (Content Weight: 5.0, Style Weight: 15.0, TV Weight: 1e1) focused on strong style transfer with significant smoothing. Some appreciated it, but others felt too much detail was lost.

How would you rate the stylization quality of this image?

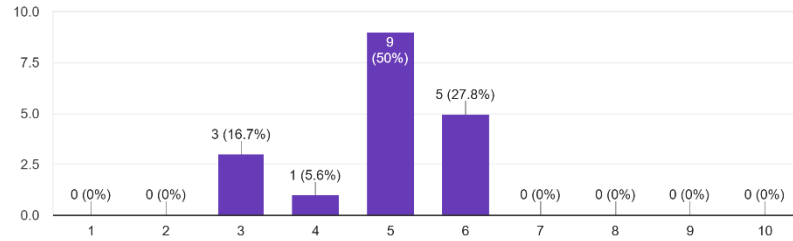
18 responses



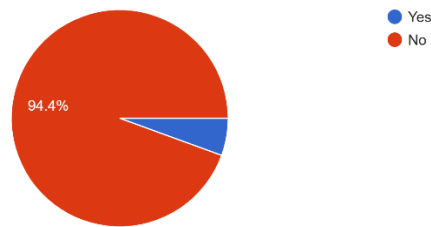
How well does this image keep the original content from the content image?
18 responses



How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content
18 responses



Did you see any problems or strange parts in the image?
18 responses



How pleasing does this image look to you?
18 responses

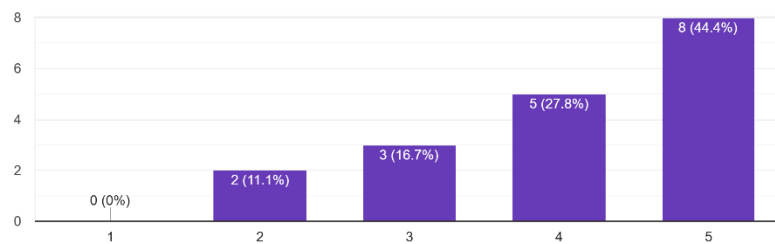


Figure (28) first, second, third, fourth, fifth survey questions and their responses for Dumoulin's experiment 8

Final choice question, which helped me choose the best parameters:

If you had to select only one image, which would it be?
18 responses

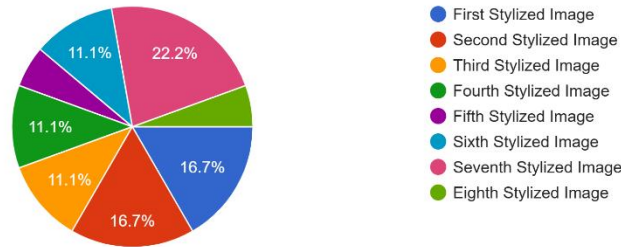


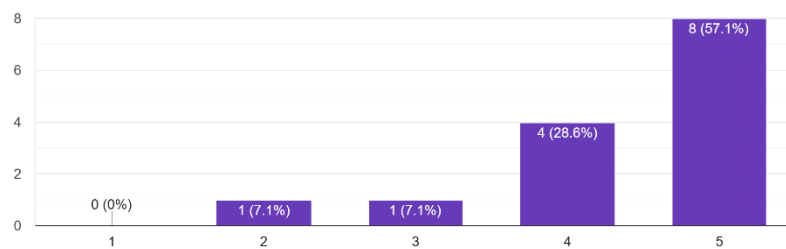
Figure (29) Final choice question for Dumoulin's experiments

AdaIN survey

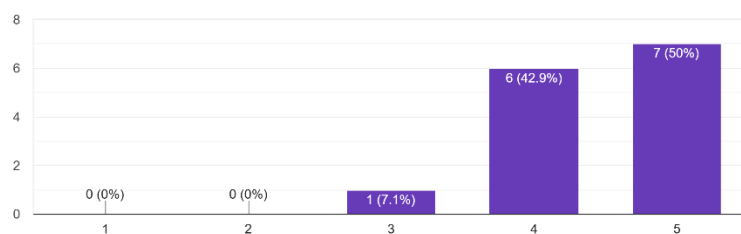
A total of 14 responses were collected for the 7 AdaIN experiments, each with different combinations of content weight, style weight, and TV weight to assess how each parameter affects the final output.

Experiment 1 (Content Weight: 1.0, Style Weight: 10.0, TV Weight: 1e-6) focused on balancing content preservation with a strong emphasis on style. The majority of participants were satisfied with the stylization, noting a good balance between content and style. However, some felt that the content was slightly compromised in favor of the style.

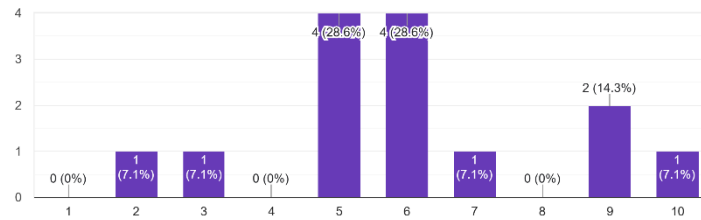
How would you rate the stylization quality of this image?
14 responses



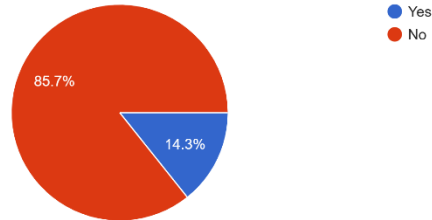
How well does this image keep the original content from the content image?
14 responses



How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content
14 responses



Did you see any problems or strange parts in the image?
14 responses



How pleasing does this image look to you?
14 responses

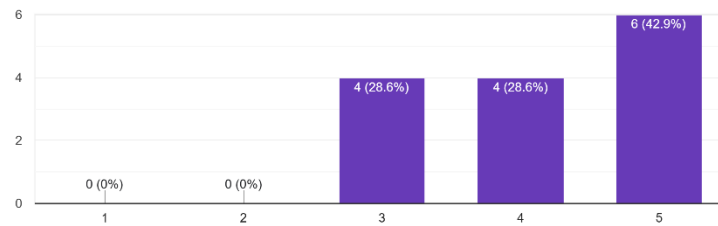
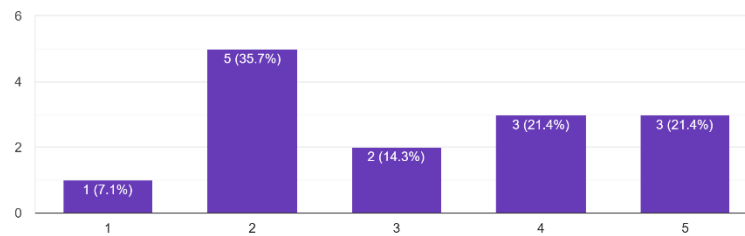


Figure (30) first, second, third, fourth, fifth survey questions and their responses for AdaIN's experiment 1

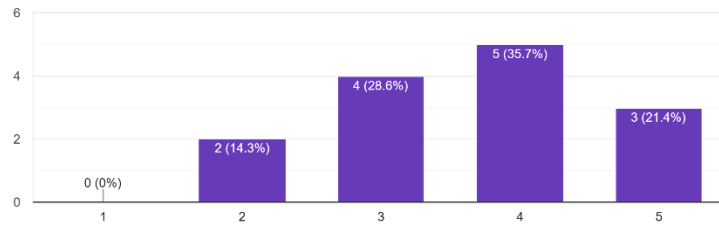
Experiment 2 (Content Weight: 5.0, Style Weight: 5.0, TV Weight: $1e-6$) aimed for equal emphasis on content and style. Responses were generally positive, with most participants appreciating the balance between the two elements, though some noted slight noise in the image.

How would you rate the stylization quality of this image?
14 responses



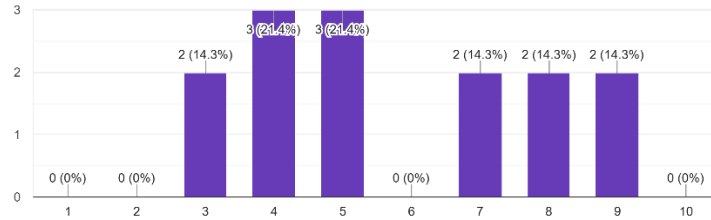
How well does this image keep the original content from the content image?

14 responses



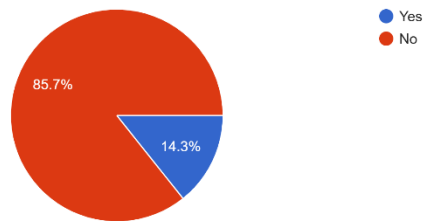
How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content

14 responses



Did you see any problems or strange parts in the image?

14 responses



How pleasing does this image look to you?

14 responses

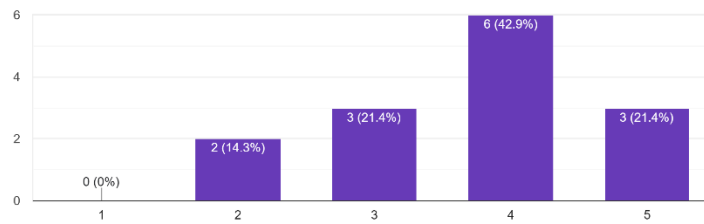
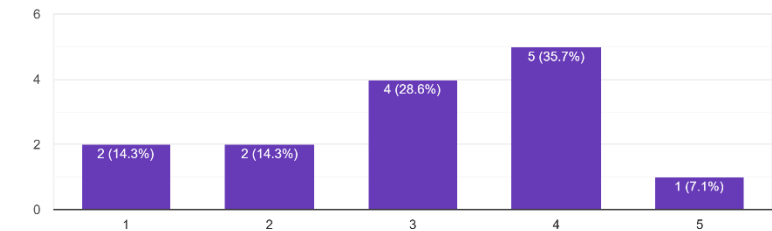


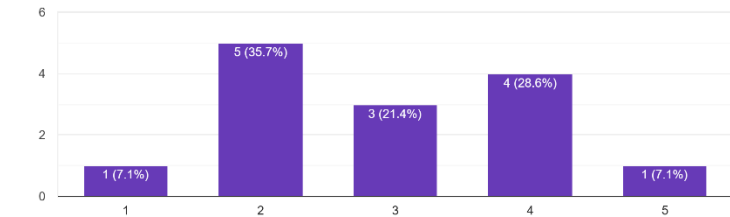
Figure (31) first, second, third, fourth, fifth survey questions and their responses for AdaIN's experiment 2

Experiment 3 (Content Weight: 1.0, Style Weight: 10.0, TV Weight: 1e-3) tested the effect of a higher TV weight, introducing more smoothing. Most participants liked the smoother output, though a few mentioned that the style was slightly reduced compared to previous experiments.

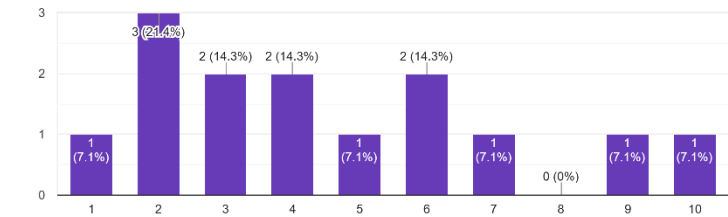
How would you rate the stylization quality of this image?
14 responses



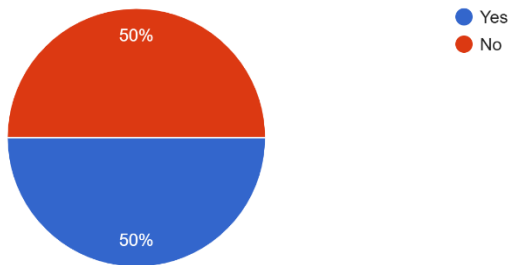
How well does this image keep the original content from the content image?
14 responses



How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content
14 responses



Did you see any problems or strange parts in the image?
14 responses



How pleasing does this image look to you?
14 responses

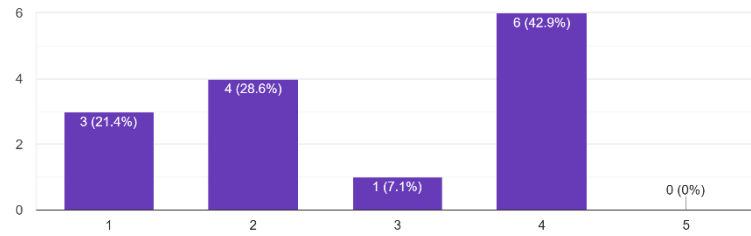
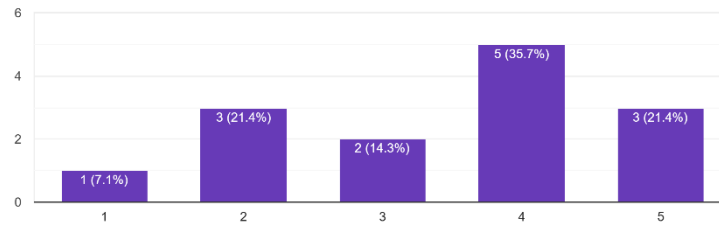


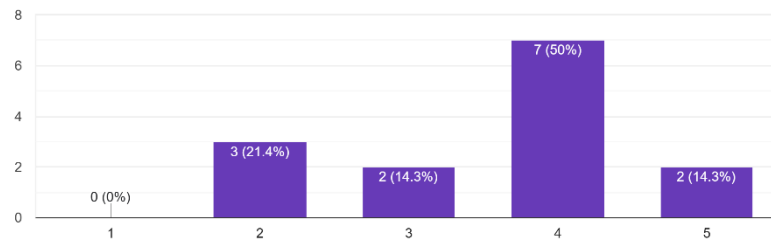
Figure (31) first, second, third, fourth, fifth survey questions and their responses for AdaIN's experiment 3

Experiment 4 (Content Weight: 5.0, Style Weight: 10.0, TV Weight: 1e-6) explored a higher emphasis on both content and style. Feedback indicated that while the content was preserved well, some participants felt the style could have been more pronounced.

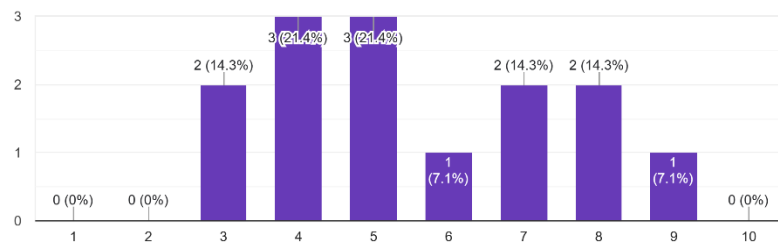
How would you rate the stylization quality of this image?
14 responses



How well does this image keep the original content from the content image?
14 responses

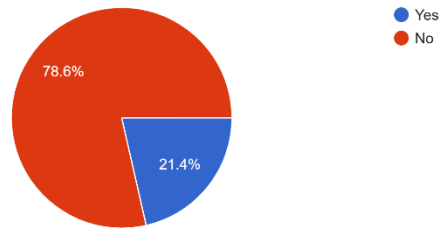


How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content
14 responses



Did you see any problems or strange parts in the image?

14 responses



How pleasing does this image look to you?

14 responses

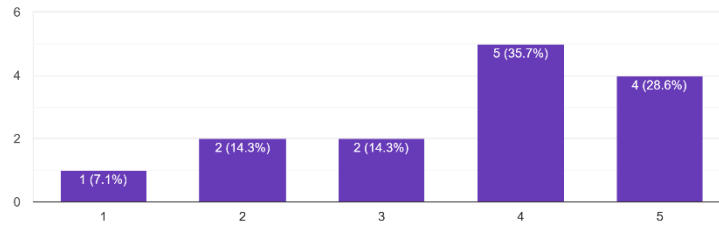


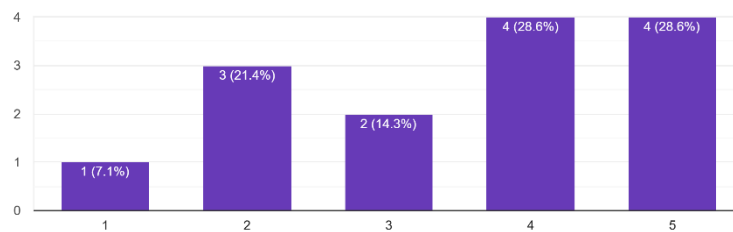
Figure (32) first, second, third, fourth, fifth survey questions and their responses for AdaIN's experiment 4

Experiment 5 (Content Weight: 1.0, Style Weight: 10.0, TV Weight: 1e1) skipping as it gave blank gray photos.

Experiment 6 (Content Weight: 1.0, Style Weight: 15.0, TV Weight: 1e-6) further emphasized the style, with minimal smoothing. Responses showed that most participants preferred the higher emphasis on style, although a few noted that content preservation could be improved.

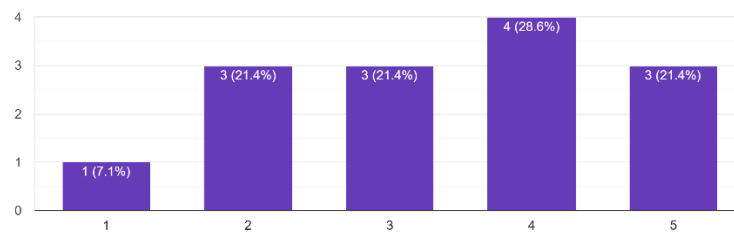
How would you rate the stylization quality of this image?

14 responses

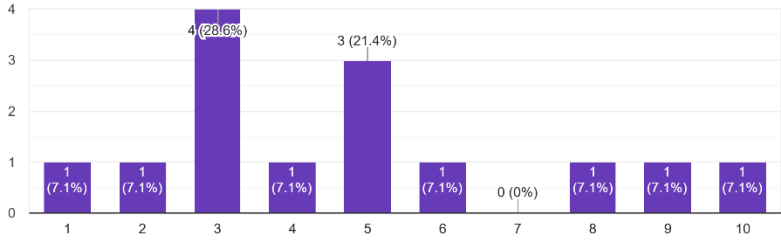


How well does this image keep the original content from the content image?

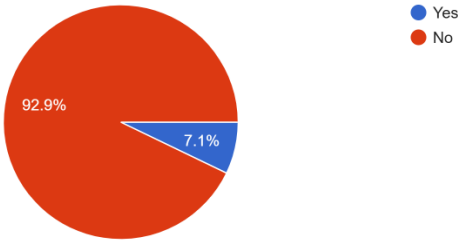
14 responses



How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content
14 responses



Did you see any problems or strange parts in the image?
14 responses



How pleasing does this image look to you?
14 responses

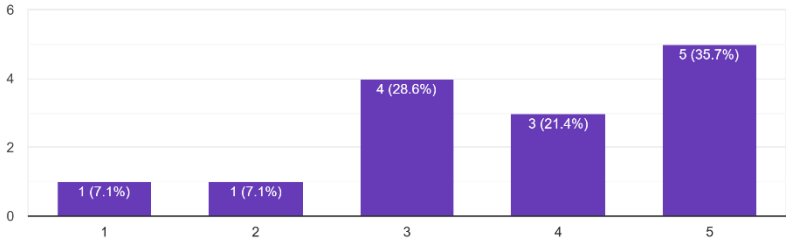
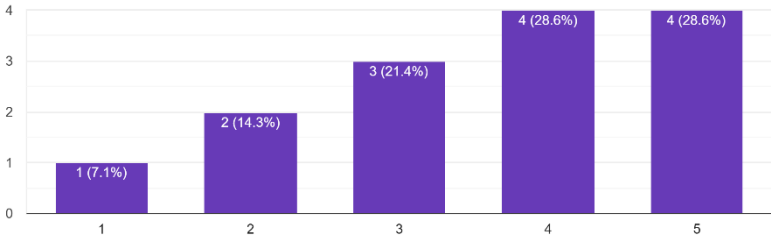


Figure (33) first, second, third, fourth, fifth survey questions and their responses for AdaIN’s experiment 6

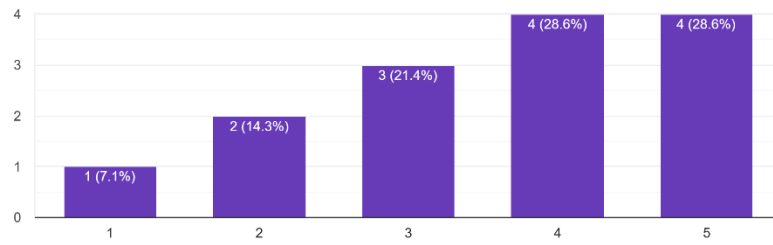
Experiment 7 (Content Weight: 5.0, Style Weight: 15.0, TV Weight: 1e-6) tested a higher style weight and content preservation. Participants generally appreciated the strong style transfer, though some felt that it came at the cost of content preservation.

How would you rate the stylization quality of this image?
14 responses



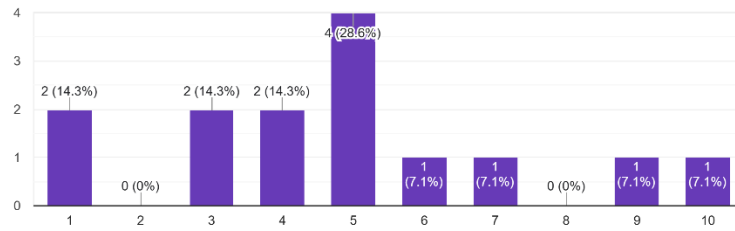
How well does this image keep the original content from the content image?

14 responses



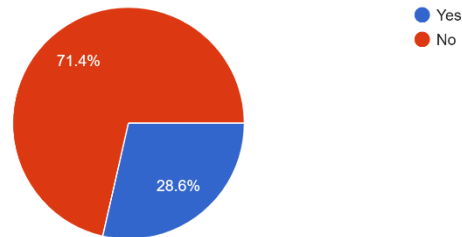
How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content

14 responses



Did you see any problems or strange parts in the image?

14 responses



How pleasing does this image look to you?

14 responses

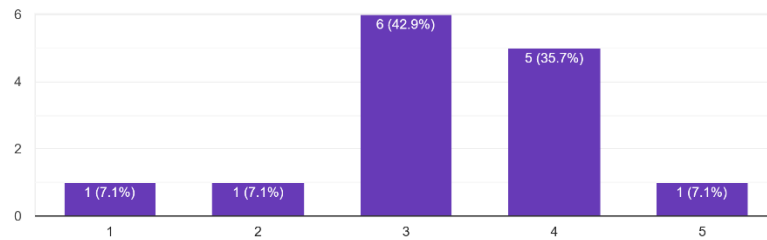


Figure (34) first, second, third, fourth, fifth survey questions and their responses for AdaIN's experiment 7

Final choice question, which helped me choose the best parameters:

If you had to select only one image, which would it be?
14 responses

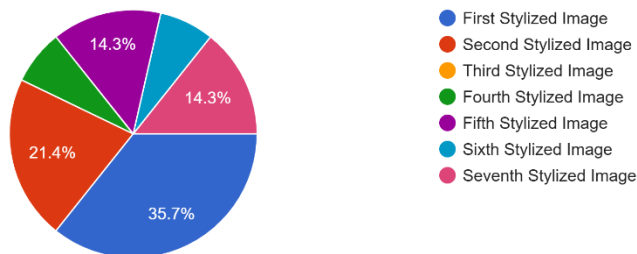


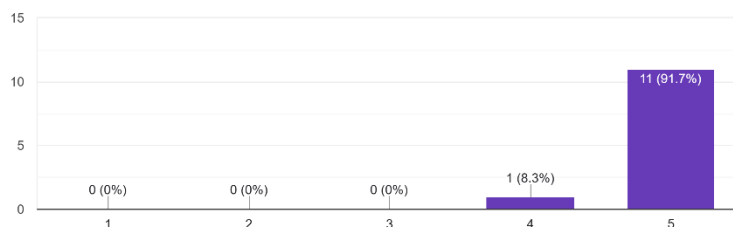
Figure (35) Final choice question for AdaIN's experiments

Gaty's survey:

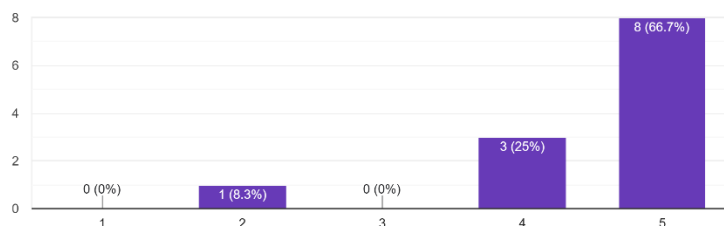
A total of 12 responses were collected for the 8 Gatys experiments. Here's a brief analysis of each:

Experiment 1 (Content Weight: $1e2$, Style Weight: $1e6$, TV Weight: $1e5$): The results were well-received, with the balance between style and content viewed positively. The image was rated highly for its pleasing appearance, though some minor issues were noted.

How would you rate the stylization quality of this image?
12 responses

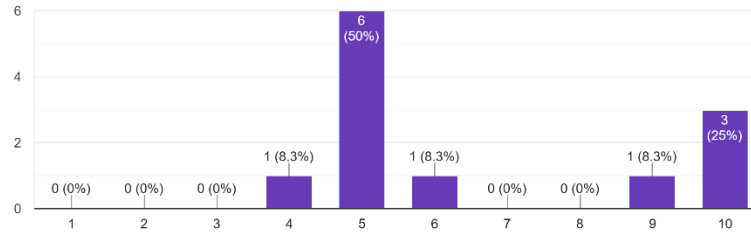


How well does this image keep the original content from the content image?
12 responses



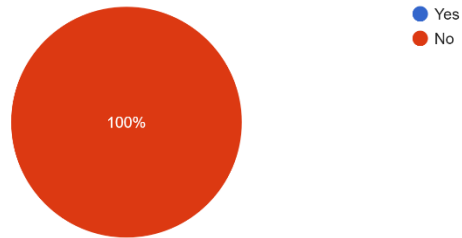
How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content

12 responses



Did you see any problems or strange parts in the image?

12 responses



How pleasing does this image look to you?

12 responses

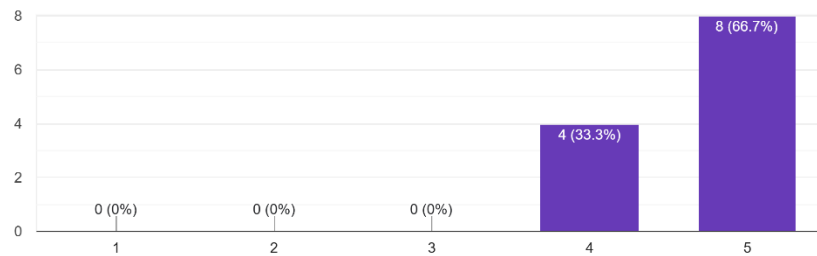
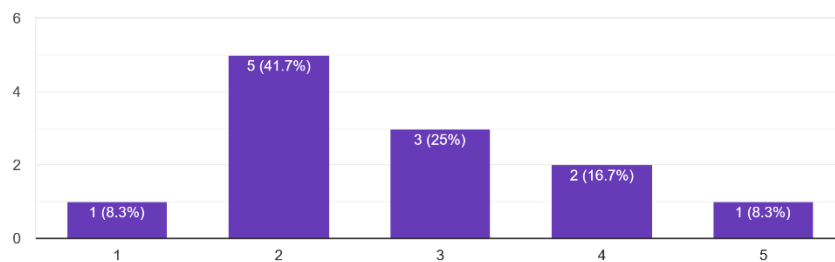


Figure (36) first, second, third, fourth, fifth survey questions and their responses for Gaty's experiment 1

Experiment 2 (Content Weight: 1e2, Style Weight: 1e4, TV Weight: 1e5): This experiment balanced content and style well. Most responses highlighted the pleasing stylization, with slight issues noted in content preservation, but the image was still liked overall.

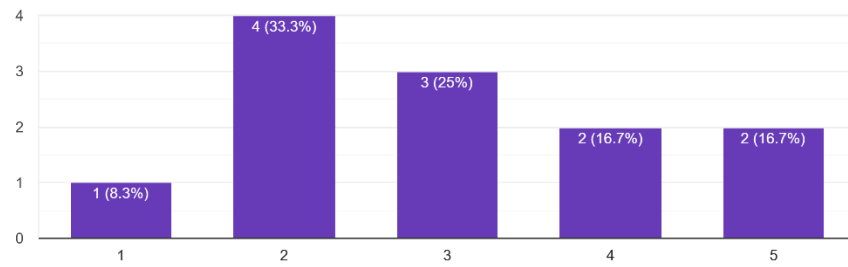
How would you rate the stylization quality of this image?

12 responses



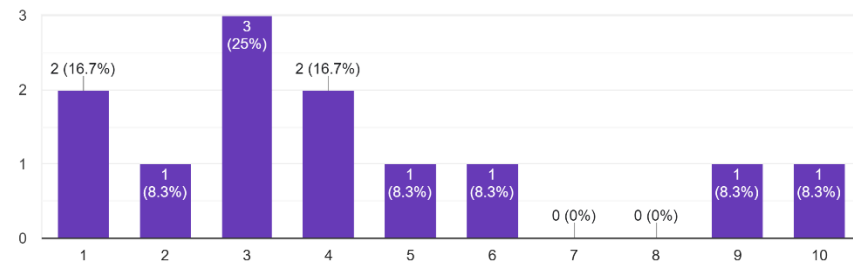
How well does this image keep the original content from the content image?

12 responses



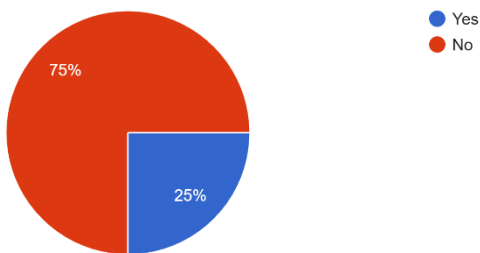
How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content

12 responses



Did you see any problems or strange parts in the image?

12 responses



How pleasing does this image look to you?

12 responses

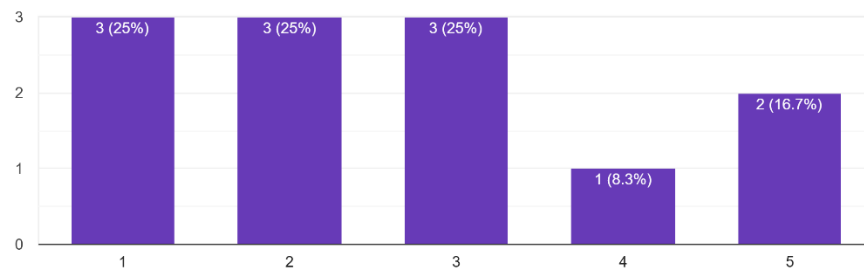
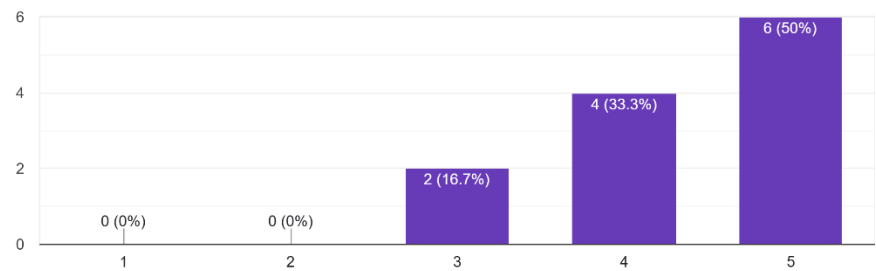


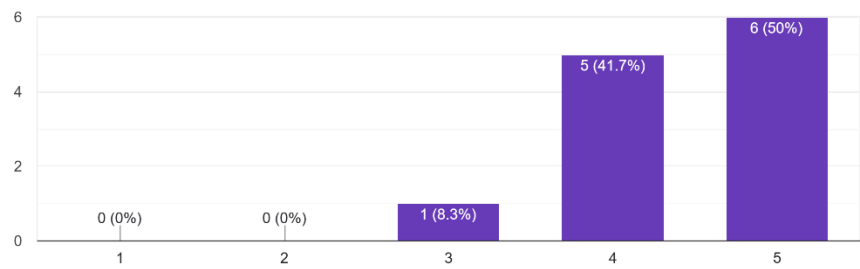
Figure (37) first, second, third, fourth, fifth survey questions and their responses for Gaty's experiment 2

Experiment 3 (Content Weight: 1e2, Style Weight: 1e8, TV Weight: 1e5): The higher style weight was favored by most respondents, but some noted a reduction in content clarity. It was still appreciated for its style, though a few participants found it too heavy on style.

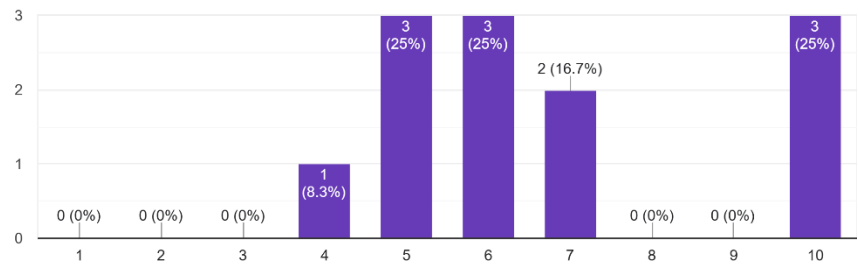
How would you rate the stylization quality of this image?
12 responses



How well does this image keep the original content from the content image?
12 responses

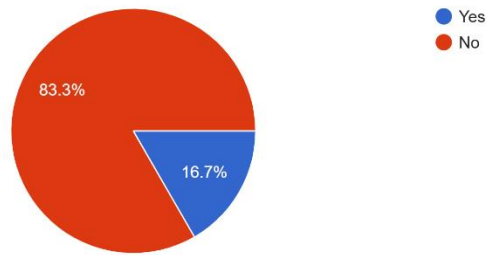


How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content
12 responses



Did you see any problems or strange parts in the image?

12 responses



How pleasing does this image look to you?

12 responses

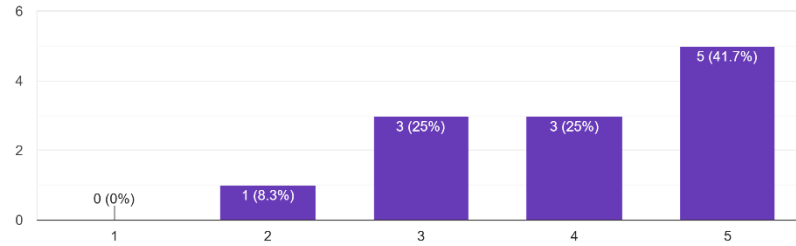
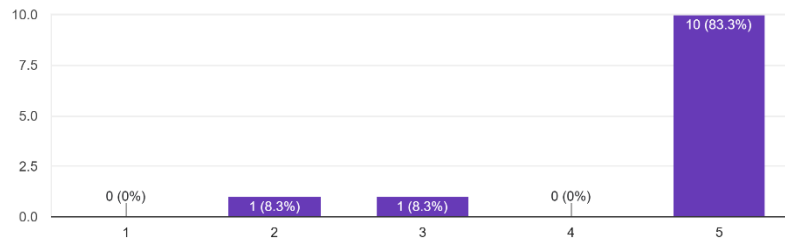


Figure (38) first, second, third, fourth, fifth survey questions and their responses for Gaty's experiment 3

Experiment 4 (Content Weight: 1e1, Style Weight: 1e6, TV Weight: 1e5): With a higher content weight, this experiment focused more on content preservation. Some feedback indicated the image was less stylized than preferred, but the content was well-preserved.

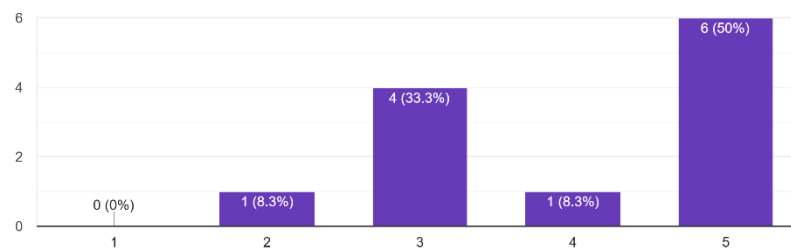
How would you rate the stylization quality of this image?

12 responses



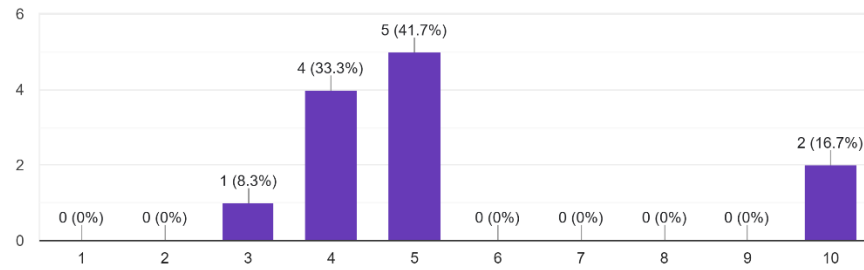
How well does this image keep the original content from the content image?

12 responses



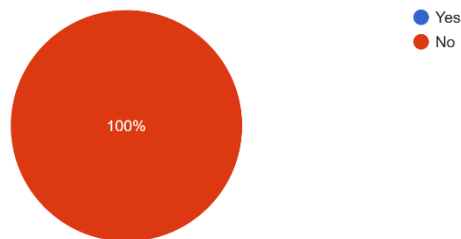
How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content

12 responses



Did you see any problems or strange parts in the image?

12 responses



How pleasing does this image look to you?

12 responses

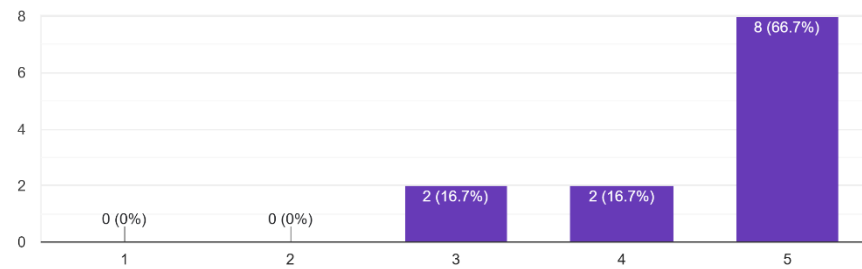
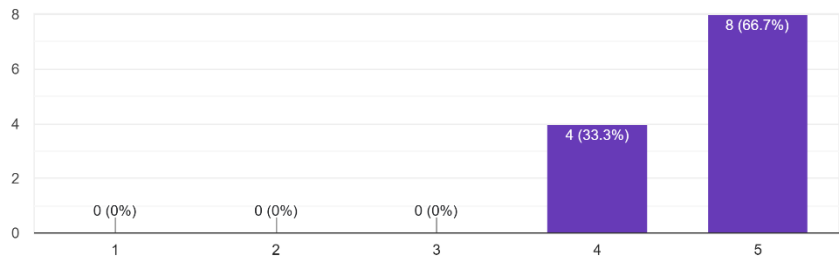


Figure (39) first, second, third, fourth, fifth survey questions and their responses for Gaty's experiment 4

Experiment 5 (Content Weight: 1e4, Style Weight: 1e6, TV Weight: 1e5) produced blank gray images and was thus excluded from evaluation.

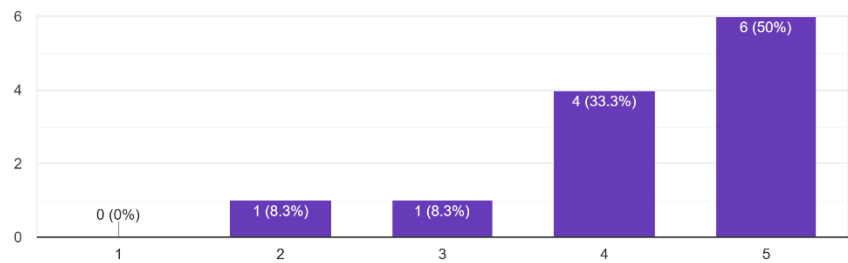
How would you rate the stylization quality of this image?

12 responses



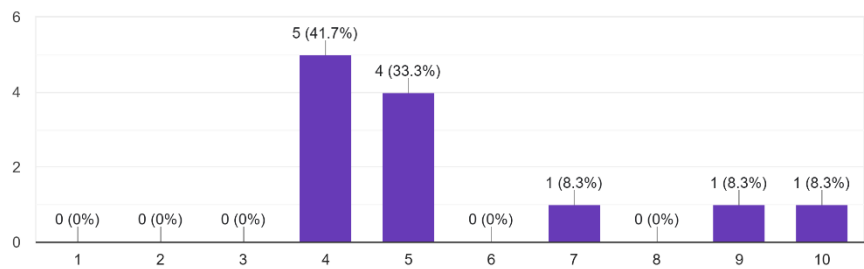
How well does this image keep the original content from the content image?

12 responses



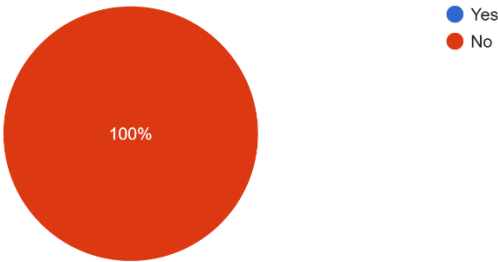
How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content

12 responses



Did you see any problems or strange parts in the image?

12 responses



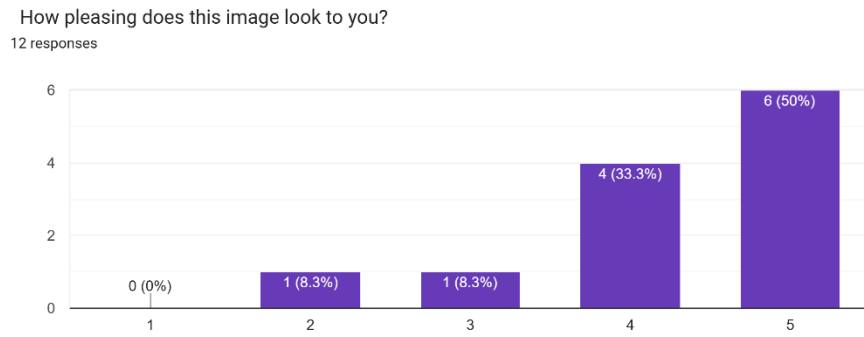
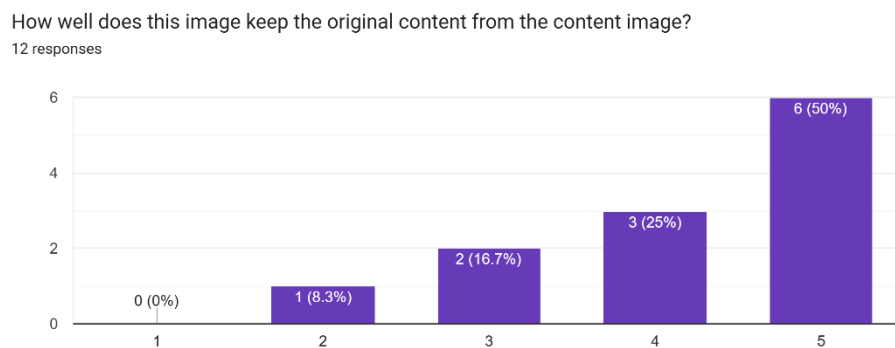
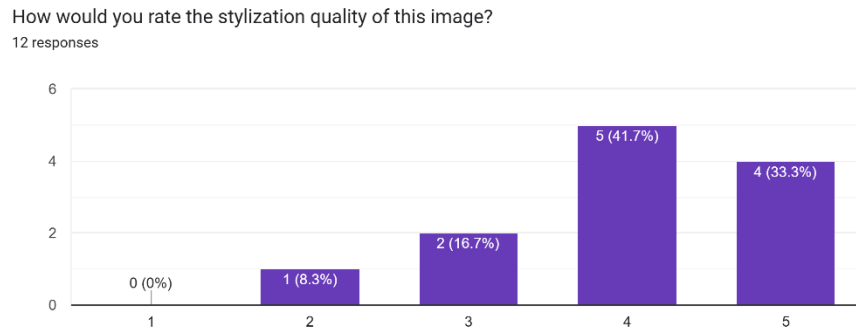
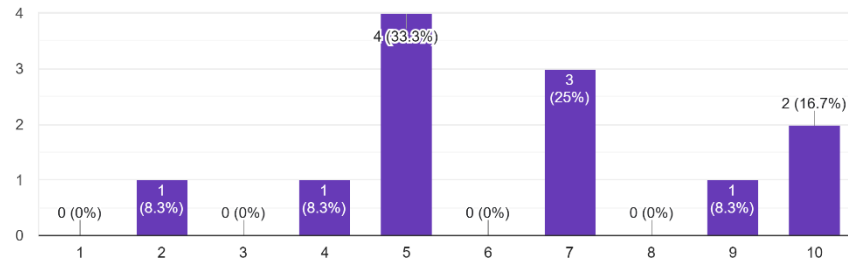


Figure (40) first, second, third, fourth, fifth survey questions and their responses for Gaty's experiment 5

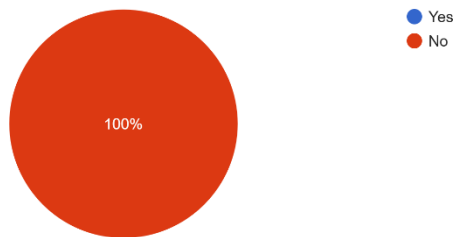
Experiment 6 (Content Weight: 1e2, Style Weight: 1e6, TV Weight: 1e3): The higher TV weight led to a smooth image with a good balance between style and content, though some participants felt the style could have been stronger.



How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content
12 responses



Did you see any problems or strange parts in the image?
12 responses



How pleasing does this image look to you?
12 responses

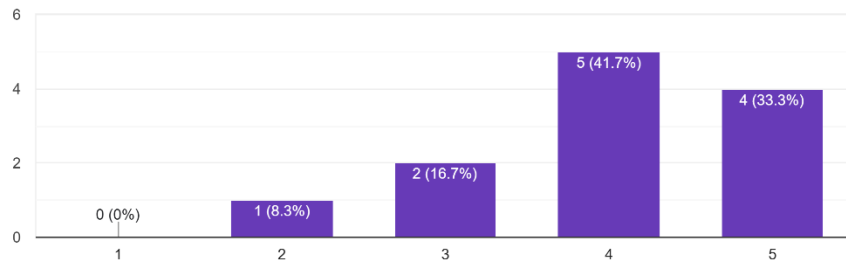
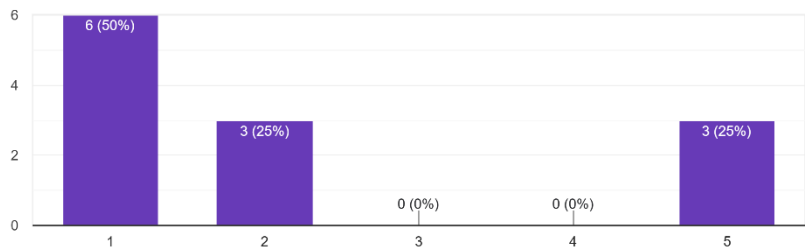


Figure (41) first, second, third, fourth, fifth survey questions and their responses for Gaty's experiment 6

Experiment 7 (Content Weight: 1e2, Style Weight: 1e6, TV Weight: 1e7): The increased TV weight resulted in a smoother image with less detail. While this was appreciated by some, it left others feeling the content wasn't preserved well enough.

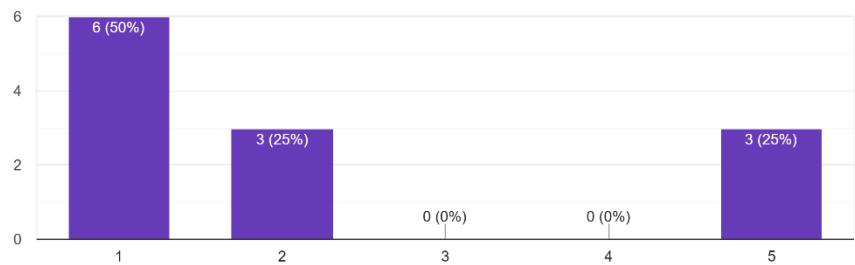
How would you rate the stylization quality of this image?

12 responses



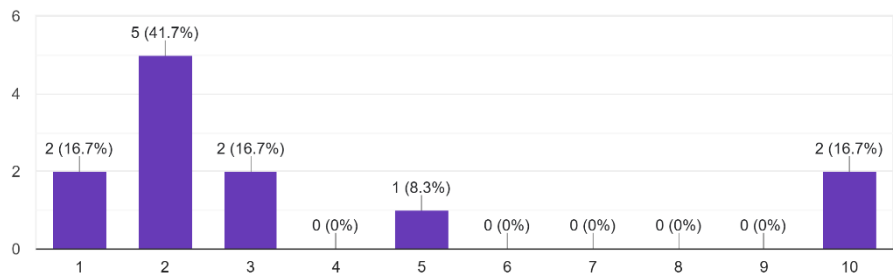
How well does this image keep the original content from the content image?

12 responses



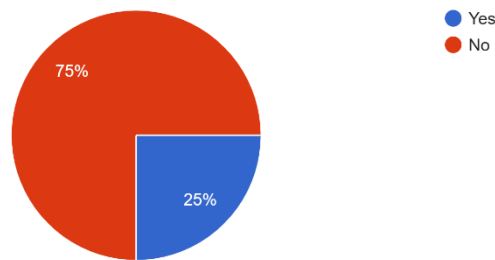
How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content

12 responses



Did you see any problems or strange parts in the image?

12 responses



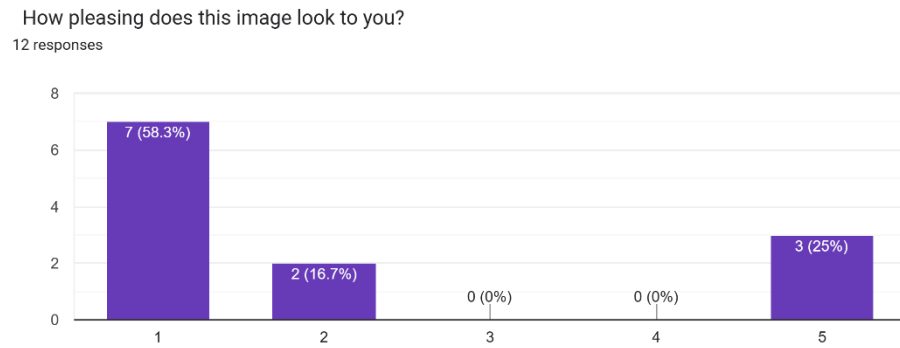
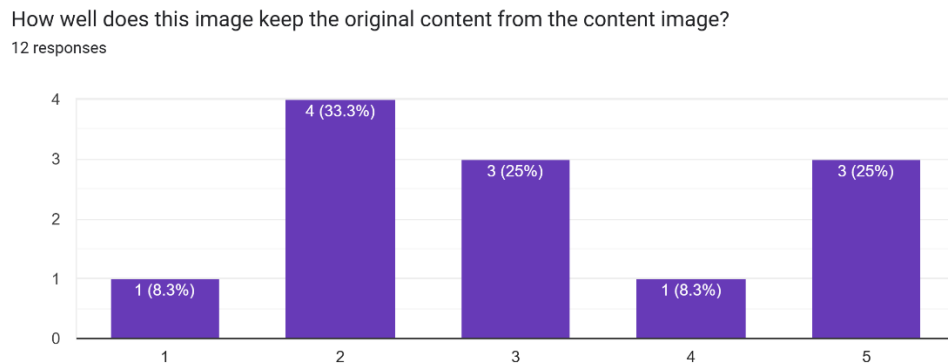
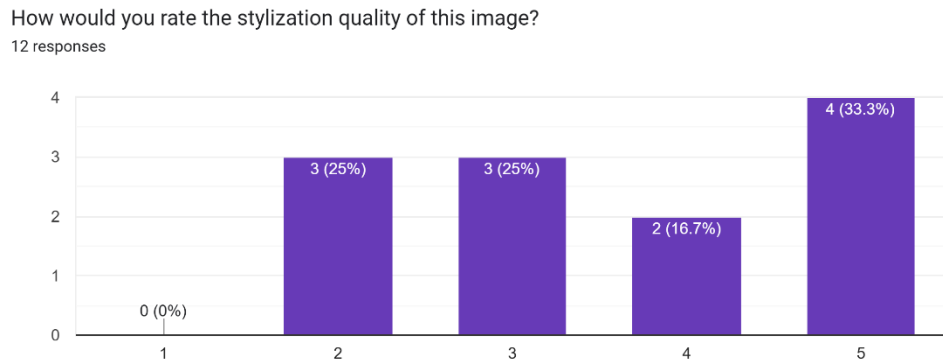


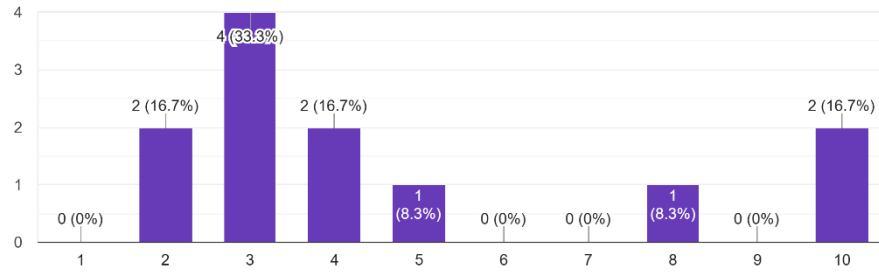
Figure (42) first, second, third, fourth, fifth survey questions and their responses for Gaty's experiment 7

Experiment 8 (Content Weight: 1e2, Style Weight: 1e4, TV Weight: 1e4): The results were balanced, but there were mixed opinions on the amount of style versus content. Some felt it was a bit too stylized for their liking, while others appreciated the balance.



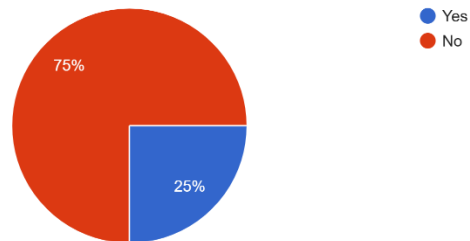
How balanced do you think the style and content are in this image? 1: Too much style 5: Perfect balance 10: Too much content

12 responses



Did you see any problems or strange parts in the image?

12 responses



How pleasing does this image look to you?

12 responses

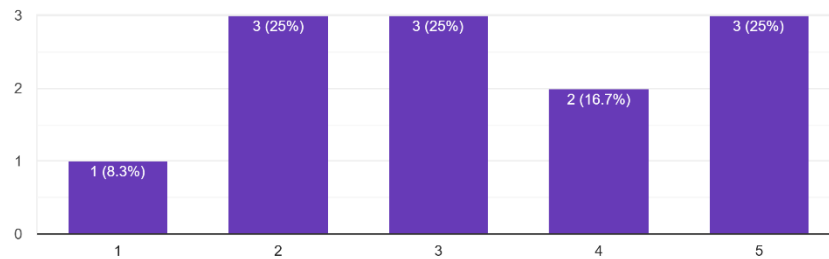


Figure (43) first, second, second, third, fourth, fifth survey questions and their responses for Gaty's experiment 8

Final choice question, which helped me choose the best parameters:

If you had to select only one image, which would it be?
12 responses

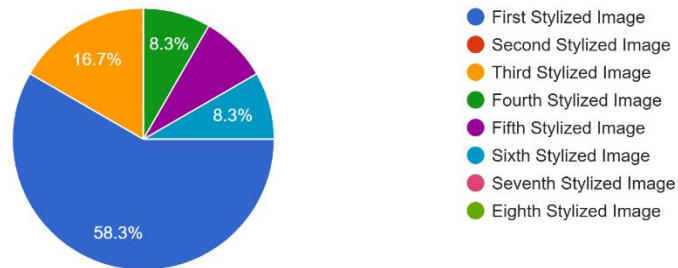
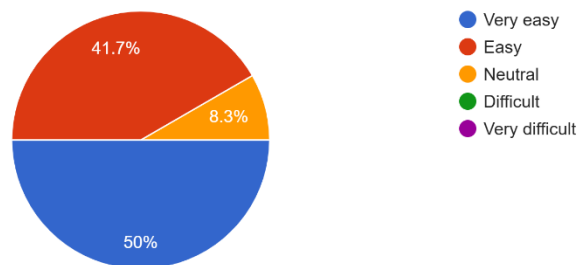


Figure (44) Final choice question for Gaty's experiments

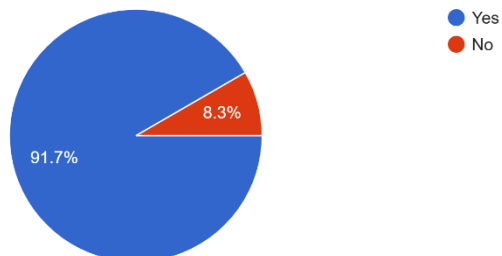
Streamlit website:

Survey to know test user friendly is the website, got 24 responders:

How easy was it to navigate the app?
24 responses

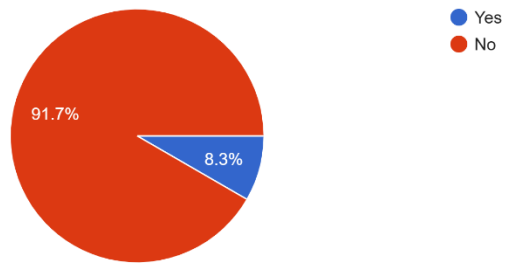


Was the app's design appealing and user-friendly?
24 responses



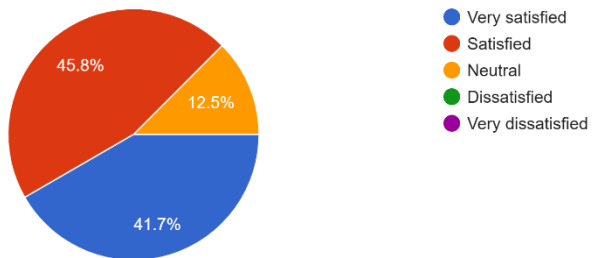
Did you encounter any technical issues while using the app?

24 responses



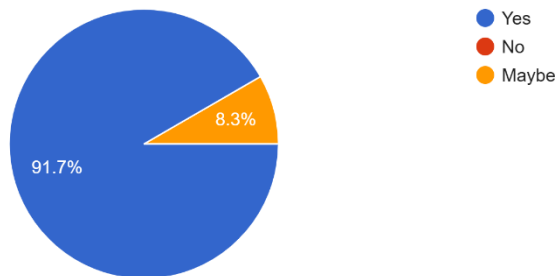
How satisfied are you with the quality of the images generated?

24 responses



Was the processing speed acceptable for generating the images?

24 responses



How would you rate the variety of styles available for Neural Style Transfer?

24 responses

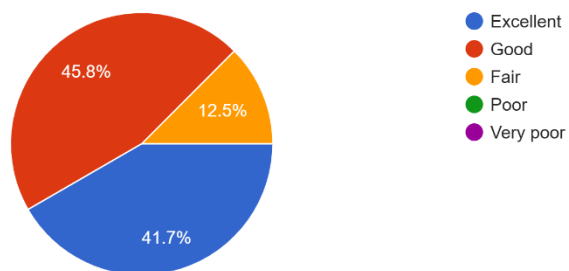


Figure (45) responds to the survey questions of the streamlit website

The feedback for the website was generally positive, with most users finding the app easy to navigate and the design appealing and user-friendly. Most respondents were very satisfied with the quality of the images generated and found the processing speed acceptable. The variety of styles available for Neural Style Transfer was rated highly, with many users finding the app's instructions clear and helpful.

5.2 Quantitative evaluation

Beyond the subjective user feedback, a more objective evaluation was performed using several key metrics to assess the models' performance quantitatively: Mean Squared Error (MSE) for content and style loss, Structural Similarity Index (SSIM), and processing time, DF/SF and LPIPS.

1. content and style loss (MSE):

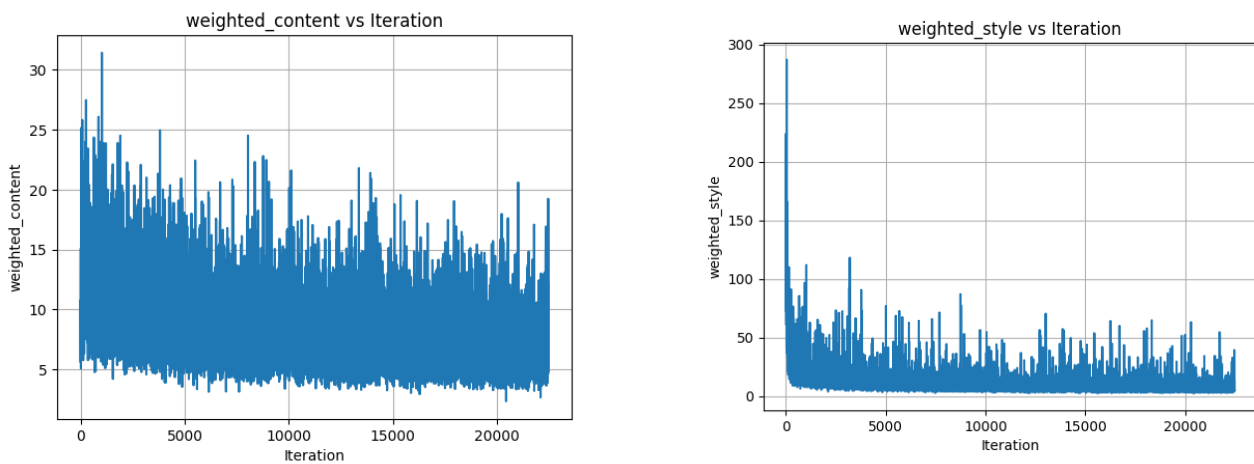


Figure (46) AdaIN's content and style loss graphs

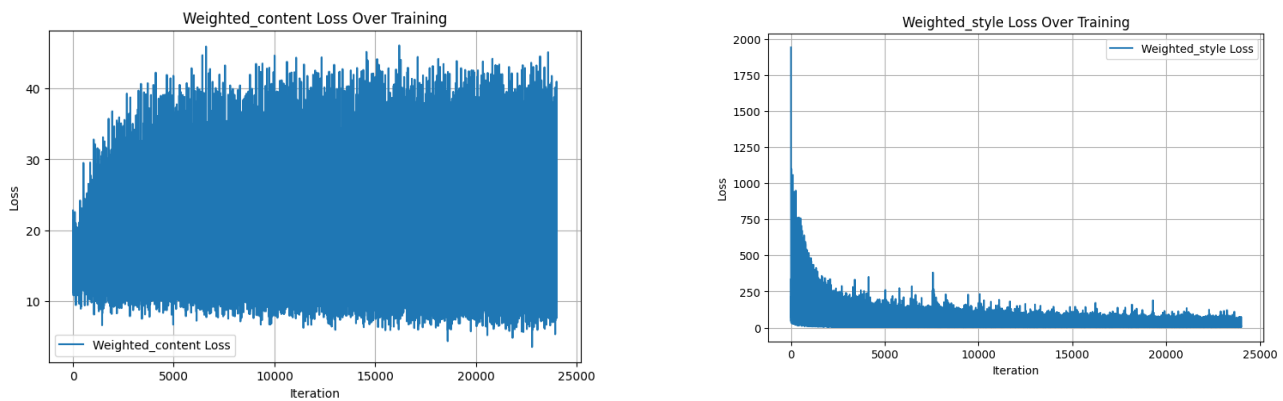


Figure (47) Dumoulin's content and style loss graphs

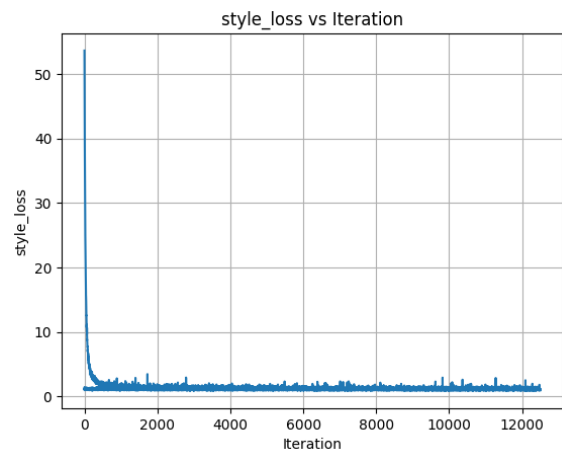
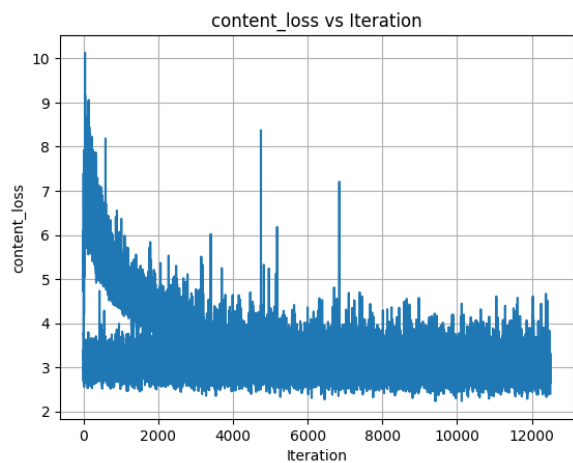


Figure (48) Johnson's content and style loss graphs

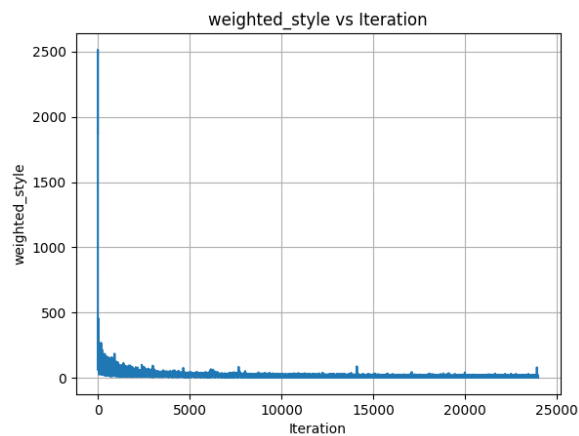
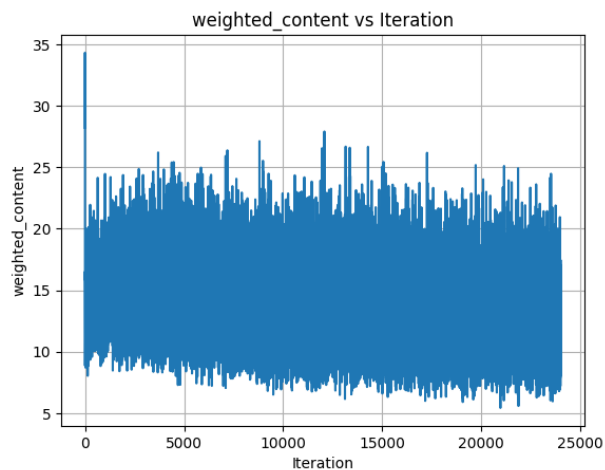


Figure (49) Dumoulin V2 content and style loss graphs

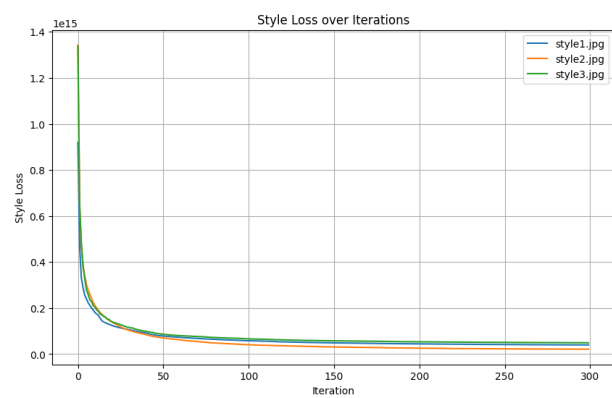
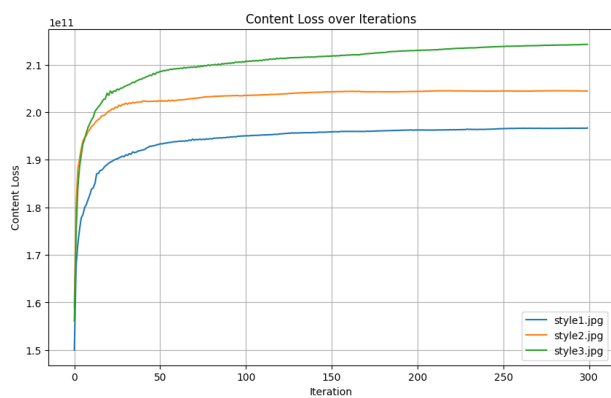


Figure (50) Gaty's content and style loss graphs

Content Loss: AdaIN exhibits moderate fluctuations in content loss, indicating reasonable content preservation with some variations. Dumoulin's content loss remains relatively steady, showing strong content preservation with minimal deviation. Johnson's content loss is slightly more variable, suggesting some challenges in preserving content. Dumoulin V2 shows the most consistent content preservation with a steady decline, reflecting effective content retention throughout. Gatys, with its high initial content loss, steadily decreases but remains higher compared to others, highlighting its primary focus on style transfer rather than content preservation.

Style Loss: AdaIN's style loss starts high but reduces significantly, demonstrating effective style transfer with minimal content impact. Dumoulin's style loss exhibits the least fluctuation, reflecting stable style transfer with some focus on content. Johnson's style loss follows a similar pattern to AdaIN, but with slightly more fluctuation, showing more pronounced style transfer. Dumoulin V2 achieves a balance between content and style loss, reflecting effective style transfer with modest content changes. Gatys, as expected, has a steep decline in style loss, indicating effective style transfer but with less focus on content fidelity.

SSIM: The Structural Similarity Index (SSIM) was done on the same content and style image for various Neural Style Transfer models reveals key differences in content preservation. Gatys et al. (2016) achieved the highest SSIM value of 0.7827, indicating that it preserved the original content with a higher degree of fidelity compared to other models. This is consistent with its focus on content and style transfer, producing images that closely resemble the original content.

In contrast, Dumoulin (2017) had a significantly lower SSIM of 0.3163, which suggests that the model focused more on style transfer at the expense of content preservation. Dumoulin V2 performed slightly better with an SSIM of 0.3698, though still not as well as Gatys et al.

Johnson et al. (2016) showed a notable improvement, with an SSIM of 0.5711, balancing content preservation and style transfer more effectively than Dumoulin models. Finally, AdaIN (2017) recorded the lowest SSIM of 0.2345, suggesting a relatively poor retention of content compared to the others.

These results highlight Gatys et al. as the most effective in terms of content preservation, while AdaIN shows more emphasis on style transfer at the cost of content.

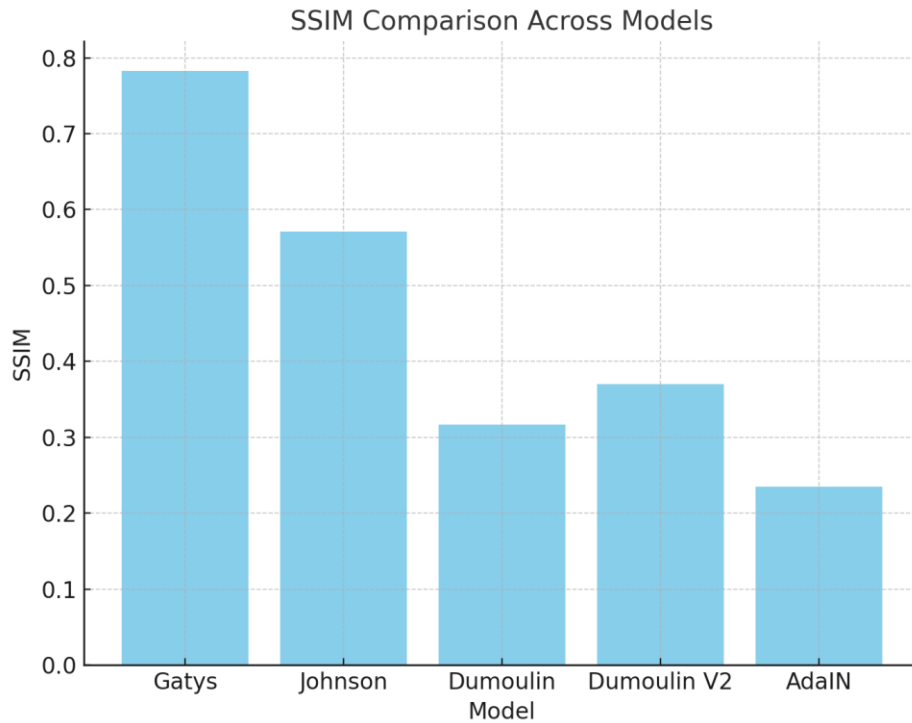


Figure (51) SSIM comparison across models

DF/SF: In terms of the DF (detail fidelity) and SF (style fidelity) metrics, the models showed varying levels of performance in preserving content and achieving the desired style. AdaIN, with the lowest SF (0.1364) and DF (0.5542), emphasizes content preservation more than style transfer. This suggests that AdaIN aims to maintain the structure of the original image, even at the expense of style adherence. Johnson, on the other hand, demonstrates a high DF (0.8638), indicating strong content preservation, while maintaining a relatively balanced SF (0.1293). This suggests that Johnson strikes a better balance between content and style. Dumoulin V2 follows with a DF of 0.7638 and an SF of 0.3151, showing good content retention and reasonable style transfer. Dumoulin also shows a strong DF (0.7165), but its SF (0.3450) indicates that style preservation plays a more significant role in its output. Finally, Gatys, with a DF of 0.5203 and SF of 0.2217, offers a balanced mix of content and style, though its detail fidelity is slightly lower. Overall, the results highlight the different trade-offs the models make between content and style preservation.

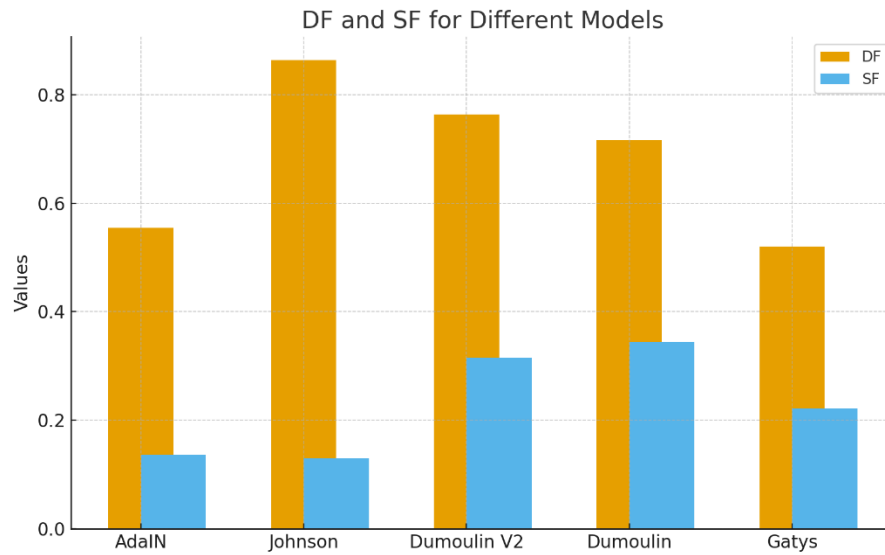


Figure (52) DF and SF comparison across models

LPIPS (Learned Perceptual Image Patch Similarity): the AdaIN model exhibited the highest value of 0.6495, indicating a greater perceptual difference from the target image, which suggests the model's struggle to preserve finer style details. Johnson's model had a lower LPIPS score of 0.4868, meaning it achieved a closer match to the style while maintaining more perceptual similarity to the content. Dumoulin V2 followed closely with a value of 0.5171, showing a moderate performance, balancing both content and style. Dumoulin produced a slightly higher value of 0.5970, indicating a slight increase in perceptual differences. Gatys had a LPIPS score of 0.5612, falling between Johnson and Dumoulin. This suggests that while Gatys maintains high style fidelity, it sacrifices some content fidelity, which is reflected in the higher LPIPS score. In conclusion, models with lower LPIPS values generally preserve content better, while higher values indicate more deviation from the reference image.

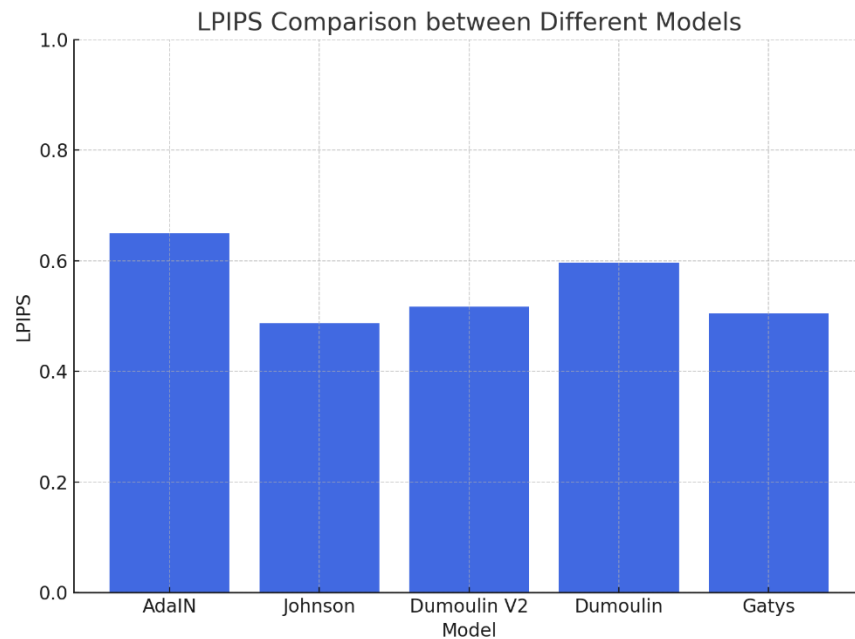


Figure (53) LPIPS comparison across models

Processing time: The training time for each model varied significantly, reflecting differences in architecture and the number of iterations. AdaIN had the longest training time, taking 8143.53 seconds (around 135 minutes) for 4 epochs with 24k iterations, which is consistent with its more complex computations. Johnson, although trained for the same number of iterations, completed training in 44 minutes (2640 seconds), indicating a more efficient processing time compared to AdaIN. Dumoulin V2 required 4261.25 seconds (71.02 minutes), while Dumoulin took the least time at 2946.35 seconds (49.11 minutes), showcasing its faster convergence.

Gatys, with only 300 iterations, had a notably shorter training time of 3481.65 seconds, but its per one content image and style image so it's not real-time.

Overall, the training time differences reflect trade-offs in model complexity, with more advanced models like AdaIN requiring more time for training while simpler models like Gatys offer quicker results, albeit with different style transfer quality.

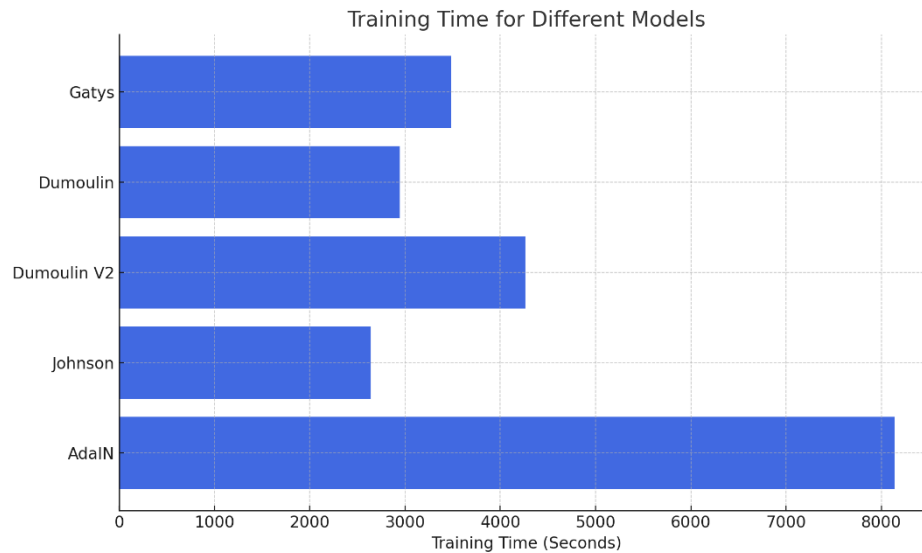


Figure (54) Processing time comparison across models

6. Conclusion (word count: 522)

This project has successfully explored and optimized Neural Style Transfer (NST) to improve its efficiency and applicability for real-time applications. By building on the foundational work of Gatys et al. (2016), we aimed to address key limitations of the original NST approach, particularly its computational inefficiency, while preserving the high-quality style and content transfer.

Through our implementation, which explored different models such as Gatys et al., Johnson et al., AdaIN, Dumoulin, and their variations, we were able to evaluate their performance based on both qualitative and quantitative metrics. The qualitative evaluation, through user feedback, revealed valuable insights into which models provide the best balance between style preservation and content fidelity. Users appreciated the flexibility of the newer models like AdaIN and Dumoulin V2 for their ability to transfer multiple styles efficiently while maintaining content integrity.

On the quantitative side, we analyzed several performance metrics, including content and style loss (MSE), SSIM, LPIPS, DF/SF, and processing time. The results showed that while Gatys et al. produced the best content preservation, it was computationally expensive, making it unsuitable for real-time applications. In contrast, models like Johnson et al., AdaIN, and Dumoulin provided much faster results with an acceptable trade-off in content and style fidelity. The training times varied significantly, with AdaIN taking the longest, while Gatys required considerably less time due to fewer iterations.

The implementation of multi-style and video style transfer was an important milestone, as it allowed us to explore more advanced features. Dumoulin V2 Multi-Style and Dumoulin V2 Video Transfer models showed significant potential for real-time applications in dynamic media, such as video production, augmented reality, and interactive media. These models enabled the combination of multiple styles within a single image and maintained temporal consistency across video frames, addressing a key challenge in NST for video content.

Despite successes, several challenges remain. First, while the models offer real-time processing, certain visual artifacts (like flickering or less intricate textures) persist, particularly in video style transfer. Additionally, as we explored the trade-off between speed and style fidelity, it became apparent that achieving both in perfect balance is an ongoing challenge. Further improvements in temporal consistency for video applications and better handling of highly detailed styles are key areas for future development.

In the future, this research could be expanded to enhance the scalability of NST, making it even more adaptable for a wider range of real-time applications, particularly in creative fields such as digital art and interactive media. Moreover, exploring deeper integration of machine learning models for adaptive style transfer, improving video coherence, and reducing the resource burden on devices with limited computational power will be key. Finally, integrating more user-centric features in the user interface, as well as leveraging cloud computing, could improve the accessibility and scalability of NST for a broader audience.

Overall, this project has laid the groundwork for practical, real-time NST applications, offering significant improvements over traditional methods and contributing to the broader field of generative art and interactive media. The advancements made here will serve as a valuable foundation for continued research in making NST faster, more scalable, and more flexible for diverse creative and practical uses.

7. References

- 1- Gatys, L.A., Ecker, A.S., & Bethge, M. (2016). Image style transfer using convolutional neural networks. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2414–2423. <https://ieeexplore.ieee.org/document/7780634>
- 2- Johnson, J., Alahi, A., & Fei-Fei, L. (2016). Perceptual losses for real-time style transfer and super-resolution. European Conference on Computer Vision (ECCV), 694–711. <https://arxiv.org/abs/1603.08155>
- 3- Huang, X., & Belongie, S. (2017). Arbitrary style transfer in real-time with adaptive instance normalization. IEEE International Conference on Computer Vision (ICCV), 1501–1510. <https://ieeexplore.ieee.org/document/8237429>
- 4- Jing, Y., Yang, Y., Feng, Z., Ye, J., Yu, Y., & Song, M. (2020). Neural style transfer: A review. IEEE Transactions on Visualization and Computer Graphics, 26(11), 3365–3385. <https://ieeexplore.ieee.org/document/8732370>
- 5- Dumoulin, V., Shlens, J., & Kudlur, M. (2017). A learned representation for artistic style. Proceedings of the International Conference on Learning Representations (ICLR).
[A Learned Representation For Artistic Style](#)
- 6- Huang, H., Wang, H., Luo, W., Ma, L., Jiang, W., Zhu, X., Li, Z., & Liu, W. (2017). Real-time neural style transfer for videos. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 7081–7090. openaccess.thecvf.com/content_cvpr_2017/papers/Huang_Real-Time_Neural_Style_CVPR_2017_paper.pdf
- 7- Chollet, F. (2018). Deep learning with Python. Manning Publications.
- 8- Wang, Z., Bovik, A.C., Sheikh, H.R., & Simoncelli, E.P. (2004). Image quality assessment: From error visibility to structural similarity. IEEE Transactions on Image Processing, 13(4), 600–612. <https://doi.org/10.1109/TIP.2003.819861>
- 9- OpenCV. (n.d.). OpenCV: Open Source Computer Vision Library. Retrieved from <https://opencv.org>
- 10- FFmpeg. (n.d.). FFmpeg: Open-source multimedia framework. Retrieved from <https://ffmpeg.org>
- 11- Waskom, M. L. (2021). Seaborn: Statistical data visualization. Journal of Open Source Software, 6(60), 3021. <https://doi.org/10.21105/joss.03021>

12- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>

13- Shao, R. (2017). COCO-WikiArt NST dataset 512x512 100000 images. Kaggle. [COCO/WikiArt NST Dataset](#)

13- Zhang, H., Xu, T., Zhang, L., Zhang, W., & Gu, S. (2023). Coarse-to-fine visual representation learning for neural style transfer. *arXiv preprint arXiv:2307.15157*. <https://arxiv.org/abs/2307.15157>